

AWS CLOUD

Documentation & Implementation Guide

3-Tier Architecture on AWS

Infrastructure: Custom VPC | Multi-AZ | 3-Tier Architecture

Domain: rxpulse.online | Region: us-east-1

Components: ALB | EC2 | ASG | Route53 | ACM | NAT Gateway | Bastion Host

Table of Contents

Table of Contents	2
Chapter 1: Introduction to Cloud Computing	7
1.1 Traditional Infrastructure and Its Problems	7
1.2 Why Cloud Computing Was Created	7
1.3 Core Advantages of Cloud Computing	8
1.3.1 Scalability	8
1.3.2 Elasticity	8
1.3.3 High Availability (HA)	8
1.3.4 Fault Tolerance	8
1.3.5 Pay-As-You-Go Model	8
1.4 The Shared Responsibility Model	9
1.5 Why Enterprises Move to Cloud	9
Chapter 2: Introduction to AWS	10
2.1 What is AWS?	10
2.2 AWS Global Infrastructure	10
2.2.1 Regions	10
2.2.2 Availability Zones (AZs)	10
2.2.3 Edge Locations	10
2.3 AWS Global Infrastructure Diagram.....	11
2.4 Why AWS is the Market Leader.....	11
Chapter 3: VPC — Virtual Private Cloud	12
3.1 What is a VPC?	12
3.2 Why VPC is Needed.....	12
3.3 CIDR Block Planning	13
3.4 Our VPC Architecture	13
3.4.1 VPC Configuration Details	13
Chapter 4: Subnets	14
4.1 Public vs. Private Subnets	14
4.2 Our 8-Subnet Design.....	14
4.3 Why Subnet Separation is Critical	15
4.3.1 Security Benefits	15
4.3.2 High Availability Benefits.....	15
4.4 VPC Architecture Diagram	15
Chapter 5: Internet Gateway	17
5.1 What is an Internet Gateway?	17
5.2 How the Internet Gateway Enables Public Access	17
5.3 Traffic Flow Through the Internet Gateway	17

Chapter 6: NAT Gateway.....	19
6.1 Why Private Instances Need NAT	19
6.2 How NAT Gateway Works.....	19
6.3 NAT Gateway Traffic Flow Diagram	20
6.4 NAT Gateway High Availability Design	20
Chapter 7: Route Tables.....	21
7.1 How Route Tables Work.....	21
7.2 Our Route Table Configuration.....	21
7.2.1 Public Route Table (for public subnets).....	21
7.2.2 Private Route Table (for private subnets).....	21
7.3 Route Table Association.....	21
Chapter 8: Elastic IP Address	23
8.1 Why Elastic IP is Needed	23
8.2 Elastic IP and NAT Gateway	23
Chapter 9: Security Groups	24
9.1 Stateful Firewall Explained	24
9.2 Security Group Rule Components	24
9.3 Our Security Group Architecture.....	24
9.3.1 ALB Security Group (Public-facing).....	24
9.3.2 Bastion Host Security Group.....	25
9.3.3 Frontend Security Group.....	25
9.3.4 Internal ALB Security Group	25
9.3.5 Backend Security Group	25
9.3.6 Database Security Group.....	26
9.4 Security Group Communication Flow	26
Chapter 10: EC2 — Elastic Compute Cloud.....	28
10.1 EC2 Instance Types	28
10.2 EC2 Components We Configure	28
10.2.1 Amazon Machine Image (AMI).....	28
10.2.2 Key Pairs	28
10.2.3 Networking Configuration for Our Instances.....	29
Chapter 11: Bastion Host.....	30
11.1 Why a Bastion Host is Needed	30
11.2 Bastion Host SSH Architecture.....	30
11.3 SSH Connection Commands.....	31
11.4 Bastion Host Best Practices	31
Chapter 12: Application Deployment and User Data Scripts	32
12.1 Why User Data is Important.....	32
12.2 Applications Used in Our Architecture	32

12.3 Frontend User Data Script (Apache + HTML)	33
12.4 Backend User Data Script (Node.js)	33
12.5 Database User Data Script (MySQL)	34
12.6 GitHub Repository Reference	34
Chapter 13: Target Groups	35
13.1 Target Group Components	35
13.2 Our Target Groups	35
13.3 Health Check Configuration	35
Chapter 14: Application Load Balancer (ALB)	37
14.1 What is Layer 7 Load Balancing?	37
14.2 ALB Components	37
14.2.1 Listeners	37
14.2.2 Listener Rules	37
14.3 ALB High Availability	37
14.4 SSL Termination	38
14.5 ALB Request Flow Diagram	38
Chapter 15: Internal Application Load Balancer	40
15.1 Why the Internal ALB is Needed	40
15.2 Internal ALB Configuration	40
15.3 Internal ALB DNS Resolution	40
Chapter 16: ALB Routing Types	42
16.1 Path-Based Routing	42
Definition	42
Real-World Use Case	42
16.2 Host-Based Routing	43
Definition	43
Real-World Use Case	43
16.3 Query String Routing	43
Definition	43
Training Example	43
16.4 HTTP Header-Based Routing	44
16.5 Source IP-Based Routing	44
Chapter 17: Auto Scaling — Launch Templates and Auto Scaling Groups	45
17.1 Why Auto Scaling is Needed	45
17.2 Launch Templates	45
Our Launch Template Configuration	45
17.3 Auto Scaling Groups (ASG)	46
17.3.1 ASG Capacity Settings	46
17.3.2 Scaling Policies	46

17.4 Multi-AZ ASG Behaviour	47
Chapter 18: ACM — AWS Certificate Manager	48
18.1 SSL/TLS Concepts	48
Why HTTPS is Critical	48
18.2 ACM Certificate Types	48
18.3 DNS Validation with Route 53	48
Chapter 19: Route 53 — DNS Management	50
19.1 What is DNS?	50
19.2 What is Route 53?	50
19.3 Hosted Zones	50
19.4 Route 53 DNS Resolution Flow Diagram	51
19.5 DNS Record Types	51
19.5.1 A Record (Address Record)	51
19.5.2 CNAME Record (Canonical Name Record)	51
19.5.3 Alias Record (AWS-Specific)	51
Chapter 20: Route 53 Routing Policies	53
20.1 Simple Routing	53
Real-World Use Case	53
20.2 Geolocation Routing	53
Real-World Use Case	53
20.3 Failover Routing	53
Real-World Use Case	53
20.4 Multi-Value Answer Routing	54
Real-World Use Case	54
Chapter 21: ACM + Route 53 + ALB Integration — Complete HTTPS Architecture	55
21.1 Component Roles in HTTPS Architecture	55
21.2 Complete HTTPS Traffic Flow	55
21.3 HTTP to HTTPS Redirect Rule	56
Chapter 22: High Availability Architecture	57
22.1 The 9s of Availability	57
22.2 How Each Layer Achieves High Availability	57
22.2.1 ALB High Availability	57
22.2.2 Frontend High Availability	57
22.2.3 Backend High Availability	57
22.2.4 Database High Availability	57
22.3 HA Architecture Summary	58
Chapter 23: Scalability Architecture	59
23.1 Horizontal vs. Vertical Scaling	59
23.2 How Our Architecture Scales	59

23.2.1 Traffic Spike Scenario	59
23.3 Scalability of Each Component.....	60
Chapter 24: Security Architecture — Defence in Depth.....	61
24.1 Security Layers in Our Architecture	61
24.2 Network Isolation Architecture	61
24.3 Zero-Trust Traffic Flow	61
Chapter 25: End-to-End 3-Tier Application Flow	62
25.1 Complete Request Lifecycle	62
25.2 Network Traffic at Each Hop.....	63
Chapter 26: Complete Practical AWS Console Implementation Guide	64
26.1 Implementation Order.....	64
26.2 Phase 1: Network Foundation	64
26.3 Phase 2: Security Groups.....	64
26.4 Phase 3: Compute Instances.....	65
26.5 Phase 4: Load Balancing.....	65
26.6 Phase 5: Auto Scaling	66
26.7 Phase 6: DNS and SSL	66
26.8 Phase 7: Validation Checklist	66
Chapter 27: Best Practices and Common Mistakes	68
27.1 Common Mistakes and How to Avoid Them	68
27.2 Architecture Best Practices.....	68
Networking.....	68
Security	68
Availability and Scaling	69
Cost Optimisation	69
Appendix A: Architecture Quick Reference	70
A.1 IP Address Reference	70
A.2 Security Group Quick Reference.....	70
A.3 Port Reference.....	70
A.4 AWS Console Navigation Quick Reference.....	71
A.5 Glossary	71

Chapter 1: Introduction to Cloud Computing

Before diving into AWS, it is essential to understand why cloud computing was created, what problems it solves, and how it fundamentally changed the way organisations build and operate technology infrastructure.

1.1 Traditional Infrastructure and Its Problems

In the traditional model, every organisation that needed to run software had to physically own and manage its own servers, networking equipment, storage arrays, and data centres. This approach — often called on-premises (on-prem) infrastructure — came with a long list of serious problems:

Problem	Impact on Business
High Capital Expenditure (CapEx)	Buying servers costs hundreds of thousands of dollars upfront before even writing a line of code.
Long Procurement Lead Times	Ordering, shipping, racking, and configuring servers could take 3–6 months.
Over-provisioning	To handle peak traffic, companies bought capacity for maximum load — meaning 90% of resources sat idle most of the time.
Under-provisioning Risk	If traffic spiked beyond capacity, the application crashed and revenue was lost.
Maintenance Burden	Physical hardware needs cooling, power, security guards, OS patches, and hardware repair teams.
No Elasticity	You cannot instantly add 100 servers for a Black Friday sale and remove them the next day.
Single Point of Failure	If your data centre floods, burns, or loses power, your entire business goes offline.
Global Reach is Expensive	To serve users in Asia and Europe, you had to build or rent data centres in each region — extremely costly.

1.2 Why Cloud Computing Was Created

Cloud computing was created to solve every single problem listed above. The fundamental idea is simple: instead of owning infrastructure, you rent it from a massive, shared pool managed by a cloud provider like Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP).

Think of it like electricity. You do not build your own power plant to power your home — you simply plug into the grid and pay for what you use. Cloud computing works the same way. You plug into AWS and pay only for the compute, storage, and networking you actually consume.

1.3 Core Advantages of Cloud Computing

1.3.1 Scalability

Scalability means the ability to handle growing amounts of work by adding more resources. In cloud computing, scalability comes in two forms:

- **Vertical Scaling (Scale Up):** Increasing the size of a single server — for example, upgrading from 2 vCPUs and 4 GB RAM to 16 vCPUs and 64 GB RAM. This is done by changing the EC2 instance type in AWS.
- **Horizontal Scaling (Scale Out):** Adding more servers to distribute the workload across multiple instances. This is the preferred cloud approach because it provides both redundancy and capacity.

Example: An e-commerce site normally runs on 2 servers. During a sale event, it automatically scales to 20 servers. After the sale, it scales back to 2. On-premises, this is impossible.

1.3.2 Elasticity

Elasticity is scalability that is automatic and real-time. It means the infrastructure automatically grows when demand increases and shrinks when demand decreases — without any human intervention. AWS achieves this through Auto Scaling Groups (ASG), which we will cover in detail later. Elasticity directly reduces cost by ensuring you never pay for resources you are not using.

1.3.3 High Availability (HA)

High Availability means designing a system so that it continues to operate even if individual components fail. In cloud computing, this is achieved by deploying resources across multiple Availability Zones (data centres) so that if one fails, another automatically takes over. The target in enterprise architecture is 99.99% uptime — which allows for only 52 minutes of downtime per year.

1.3.4 Fault Tolerance

Fault Tolerance goes one step beyond high availability. A fault-tolerant system continues operating without any service interruption even when a component fails. This is achieved through redundancy — having duplicate components at every layer so that failure of any single component has zero impact on end users.

1.3.5 Pay-As-You-Go Model

In cloud computing, you pay only for what you use, measured in seconds or hours. There are no upfront hardware purchases, no long-term commitments required, and no wasted capacity costs. This transforms infrastructure spend from Capital Expenditure (CapEx) — buying things — to Operational Expenditure (OpEx) — paying for ongoing service. This is a massive financial advantage for businesses.

1.4 The Shared Responsibility Model

The AWS Shared Responsibility Model defines who is responsible for what in a cloud environment. Understanding this is critical for security compliance.

Responsibility Layer	Who is Responsible
Physical security of data centres	AWS — you never touch hardware
Hypervisor and virtualisation layer	AWS
Network infrastructure (switches, routers)	AWS
Operating System patches on EC2	Customer — you manage your own OS
Application security and code	Customer
Data encryption and access control	Customer
IAM roles, policies, and permissions	Customer
Security Group configuration	Customer
Network ACL configuration	Customer

Key Principle: AWS secures the cloud. You secure what is IN the cloud.

AWS is responsible for: Hardware, Networking, Facilities, Managed Services.

You are responsible for: OS, Applications, Data, Access Control, Encryption.

1.5 Why Enterprises Move to Cloud

- Speed to Market: Launch new applications in minutes instead of months.
- Global Reach: Deploy to 30+ AWS regions worldwide instantly.
- Innovation: Access managed AI, analytics, and developer tools without building them.
- Cost Optimisation: Eliminate data centre capital costs and maintenance overheads.
- Disaster Recovery: Replicate across regions for business continuity at a fraction of the cost.
- Security Compliance: AWS is certified for PCI-DSS, HIPAA, SOC 2, ISO 27001, and many more standards.
- Focus on Business: Engineering teams focus on product features instead of infrastructure management.

Chapter 2: Introduction to AWS

Amazon Web Services (AWS) is the world's most comprehensive and broadly adopted cloud platform. Launched commercially in 2006, AWS pioneered cloud computing and today offers over 200 fully featured services from data centres globally.

2.1 What is AWS?

AWS is a subsidiary of Amazon.com that provides on-demand cloud computing platforms and APIs to individuals, companies, and governments on a metered pay-as-you-go basis. AWS allows organisations to replace upfront capital infrastructure expenses with low variable costs that scale with their business.

In simple terms: AWS is a massive collection of computing resources (servers, storage, networking, databases, AI tools) that you can rent over the internet and pay only for what you use.

2.2 AWS Global Infrastructure

2.2.1 Regions

An AWS Region is a physical geographic location in the world where AWS has clustered its data centres. Each region is completely independent and isolated from other regions. As of 2024, AWS has 33+ regions worldwide including us-east-1 (Northern Virginia), eu-west-1 (Ireland), ap-southeast-1 (Singapore), and many more.

Why regions matter: Deploying your application in a region close to your users reduces latency. Regulations may require data to remain in specific countries. Using multiple regions provides disaster recovery capabilities.

2.2.2 Availability Zones (AZs)

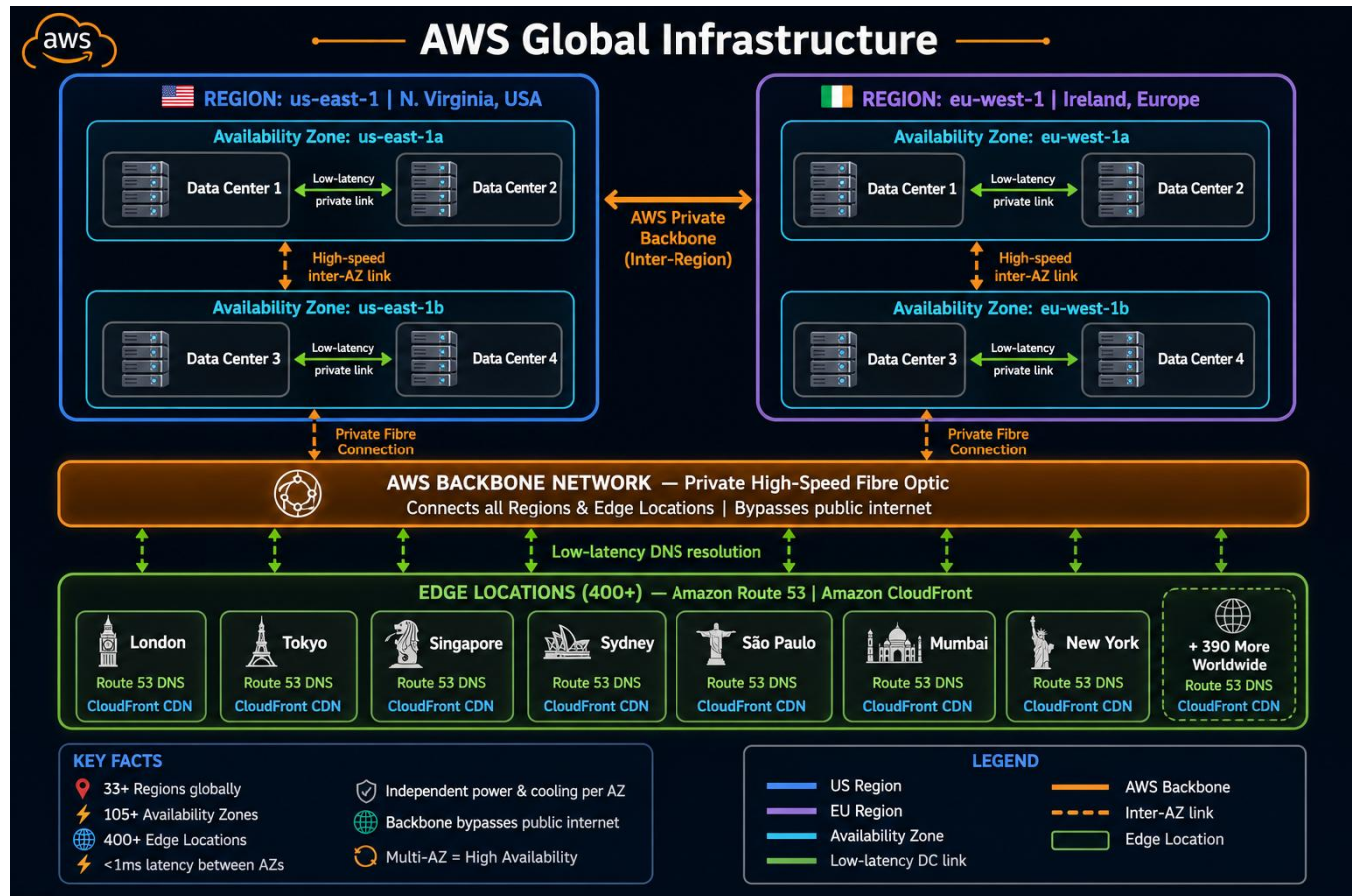
An Availability Zone is one or more discrete data centres within a region, each with redundant power, networking, and connectivity. AZs within a region are physically separated — typically by many kilometres — but connected with high-bandwidth, low-latency networking. Each region has a minimum of 3 AZs (some have 6).

In our training infrastructure, we use us-east-1a and us-east-1b — two separate AZs within the us-east-1 region. Deploying across both AZs ensures that if one data centre has an issue, the other continues serving traffic.

2.2.3 Edge Locations

Edge Locations are AWS data centres used primarily by CloudFront (CDN) and Route 53 (DNS) to serve content and resolve DNS queries with the lowest possible latency. There are 400+ Edge Locations worldwide — far more than the number of regions — positioned in major cities globally. When a user in London queries rxpulse.online, Route 53 responds from the nearest Edge Location rather than travelling all the way to us-east-1.

2.3 AWS Global Infrastructure Diagram



2.4 Why AWS is the Market Leader

- Launched in 2006 — 8 years ahead of most competitors, giving it the most mature and complete service catalogue.
- Holds approximately 32% of the global cloud market share (2024).
- 200+ services: compute, storage, database, networking, AI/ML, security, analytics, and more.
- Trusted by millions of organisations including Netflix, Airbnb, NASA, the US Government, and major banks.
- Industry-leading compliance: 143 security standards and certifications.
- The most extensive partner and training ecosystem in the cloud industry.

Chapter 3: VPC — Virtual Private Cloud

A Virtual Private Cloud (VPC) is the foundation of every AWS architecture. Understanding VPC deeply is critical because every other service — EC2, ALB, RDS, Lambda — runs inside a VPC. It is the first thing you create when building any AWS infrastructure.

3.1 What is a VPC?

A VPC is a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define. Think of it as your own private data centre inside AWS — with your own IP address range, your own subnets, your own routing rules, and your own security controls. The outside world cannot access anything inside your VPC unless you explicitly allow it.

Analogy: A VPC is like a gated office building.

The building has its own internal phone network (private IPs).

People inside can call each other freely.

To call outside, they go through a reception desk (Internet Gateway / NAT).

Visitors from outside must enter through the guarded front door (public subnet).

Internal offices (private subnets) are not directly accessible from outside.

3.2 Why VPC is Needed

- **Network Isolation:** Resources in your VPC cannot be accessed by other AWS customers or the public internet unless you configure it explicitly.
- **Security:** You control all inbound and outbound traffic using security groups and network ACLs.
- **Custom IP Addressing:** You define your own IP address range (CIDR block), just like in a traditional on-premises network.
- **Compliance:** Regulations like PCI-DSS, HIPAA, and GDPR require that systems containing sensitive data are network-isolated.
- **Traffic Segmentation:** Different application tiers (web, app, database) can be placed in different network segments.

3.3 CIDR Block Planning

CIDR (Classless Inter-Domain Routing) notation defines a range of IP addresses. The notation is: IP_Address/Prefix_Length. The prefix length determines how many addresses are in the range.

CIDR Block	Total IP Addresses
10.0.0.0/8	16,777,214 addresses (huge — rarely used for a single VPC)
10.0.0.0/16	65,534 addresses (common for enterprise VPCs — our training uses this)
10.0.0.0/24	254 addresses (common for individual subnets)
10.0.1.0/24	254 addresses (next subnet block)

Our VPC uses 10.0.0.0/16. This gives us the range from 10.0.0.0 to 10.0.255.255 — 65,536 addresses — which we subdivide into 8 subnets for our 3-tier architecture.

3.4 Our VPC Architecture

3.4.1 VPC Configuration Details

Parameter	Value
VPC Name	production-vpc
CIDR Block	10.0.0.0/16
Region	us-east-1 (Northern Virginia)
Availability Zones Used	us-east-1a and us-east-1b
DNS Hostnames	Enabled
DNS Resolution	Enabled
Total Subnets	8 (2 public + 6 private across 2 AZs)

Chapter 4: Subnets

A subnet (short for subnetwork) is a range of IP addresses within your VPC. Subnets allow you to segment your VPC into smaller, purpose-specific network segments. Each subnet lives entirely within a single Availability Zone — it cannot span multiple AZs.

4.1 Public vs. Private Subnets

The key difference between a public and private subnet is not the name — it is the routing configuration:

Feature	Public Subnet Private Subnet
Route to Internet	Has a route to Internet Gateway (IGW) No route to IGW
Instance Public IPs	Instances can have public IPs Instances never have public IPs
Internet Accessibility	Resources are directly reachable from internet Resources are isolated from internet
Outbound Internet Access	Direct via IGW Only via NAT Gateway (if configured)
Use Case	Bastion Host, NAT Gateway, Public ALB App servers, databases, internal ALBs

4.2 Our 8-Subnet Design

Our training architecture uses 8 subnets carefully designed across 2 Availability Zones to create a robust, highly available 3-tier network:

Subnet Name	CIDR Block	AZ / Purpose
public-subnet-1a	10.0.1.0/24	us-east-1a — Bastion Host, NAT Gateway, Public ALB
public-subnet-1b	10.0.2.0/24	us-east-1b — Redundant public subnet for HA
frontend-private-1a	10.0.3.0/24	us-east-1a — Frontend EC2 instances (Nginx/Apache)
frontend-private-1b	10.0.4.0/24	us-east-1b — Frontend EC2 instances (redundancy)
backend-private-1a	10.0.5.0/24	us-east-1a — Backend EC2 instances (Node.js/Flask)
backend-private-1b	10.0.6.0/24	us-east-1b — Backend EC2 instances (redundancy)
db-private-1a	10.0.7.0/24	us-east-1a — MySQL Database instance
db-private-1b	10.0.8.0/24	us-east-1b — Database standby (for future HA)

4.3 Why Subnet Separation is Critical

4.3.1 Security Benefits

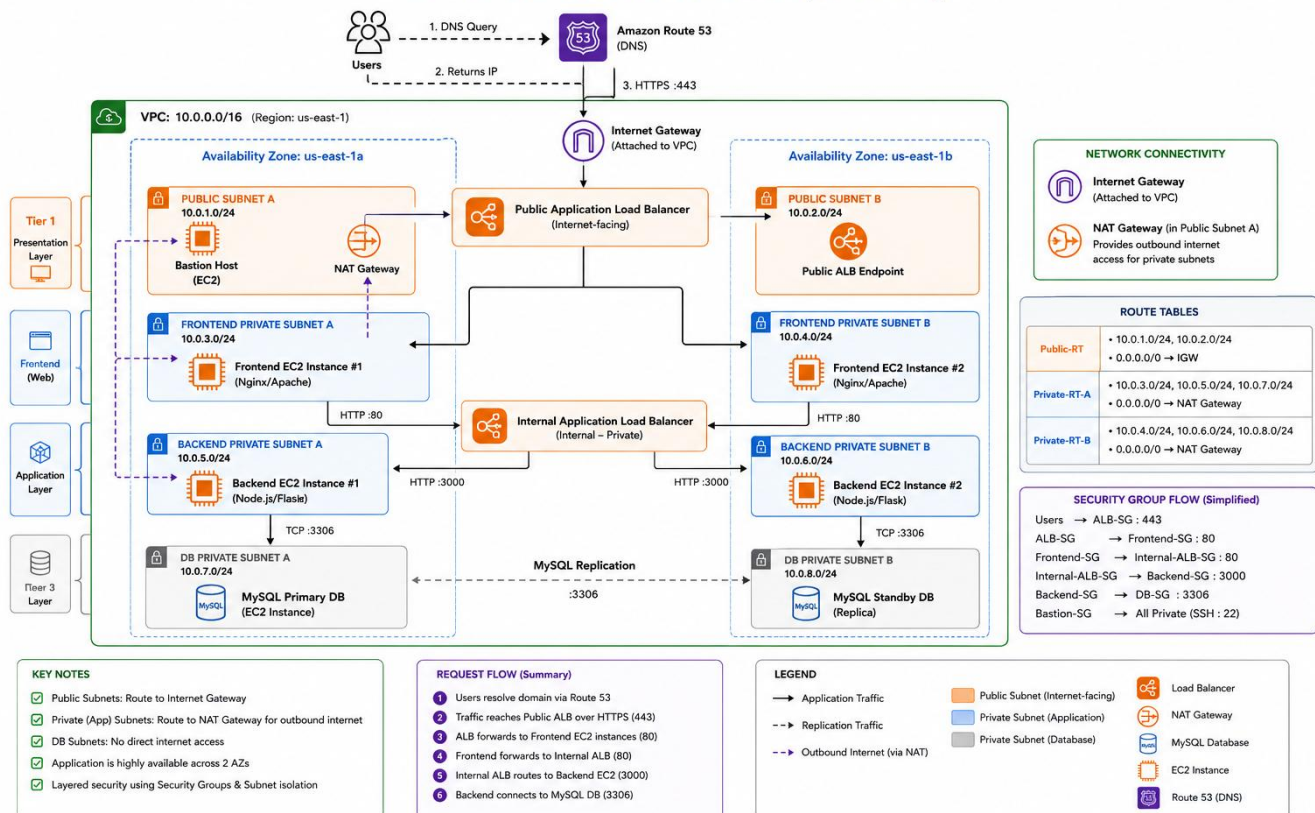
- Database servers are in a completely separate subnet with no inbound rules except from the backend subnet on port 3306. An attacker who somehow compromises the public subnet cannot reach the database because there is no route.
- Backend servers are in private subnets — they cannot be accessed from the internet at all, only from the Internal ALB.
- Frontend servers receive traffic only from the public ALB — not directly from end users.

4.3.2 High Availability Benefits

- Spreading subnets across 2 AZs means the application survives a complete failure of one AWS data centre.
- The ALB distributes traffic across both AZs, so if AZ-1a fails, traffic automatically flows only to AZ-1b instances.

4.4 VPC Architecture Diagram

4.4 – 3-Tier VPC Architecture (us-east-1)



AWS Console Implementation Steps

- Step 1: Navigate to AWS Console > VPC > Your VPCs > Create VPC
- Step 2: Select 'VPC only' (not 'VPC and more' for learning purposes)
- Step 3: Name: production-vpc
- Step 4: IPv4 CIDR Block: 10.0.0.0/16
- Step 5: Tenancy: Default
- Step 6: Click 'Create VPC'
- Step 7: Navigate to VPC > Subnets > Create Subnet
- Step 8: Select your VPC, set Subnet Name, select AZ, enter CIDR (e.g. 10.0.1.0/24)
- Step 9: Repeat for all 8 subnets
- Step 10: For public subnets: Select subnet > Actions > Edit subnet settings > Enable auto-assign public IPv4

Chapter 5: Internet Gateway

An Internet Gateway (IGW) is a horizontally scaled, redundant, and highly available VPC component that allows communication between your VPC and the public internet. Without an Internet Gateway, nothing in your VPC can communicate with the outside world — in either direction.

5.1 What is an Internet Gateway?

The Internet Gateway serves two purposes: (1) it provides a target in your route tables for internet-routable traffic, and (2) it performs Network Address Translation (NAT) for instances with public IP addresses. It is completely managed by AWS — you never need to worry about its availability or capacity.

Key Fact: An Internet Gateway is NOT a bottleneck. It is horizontally scaled by AWS automatically.

One IGW per VPC — you cannot attach more than one IGW to a VPC.

An IGW is stateful — if traffic is allowed in, the response is automatically allowed out.

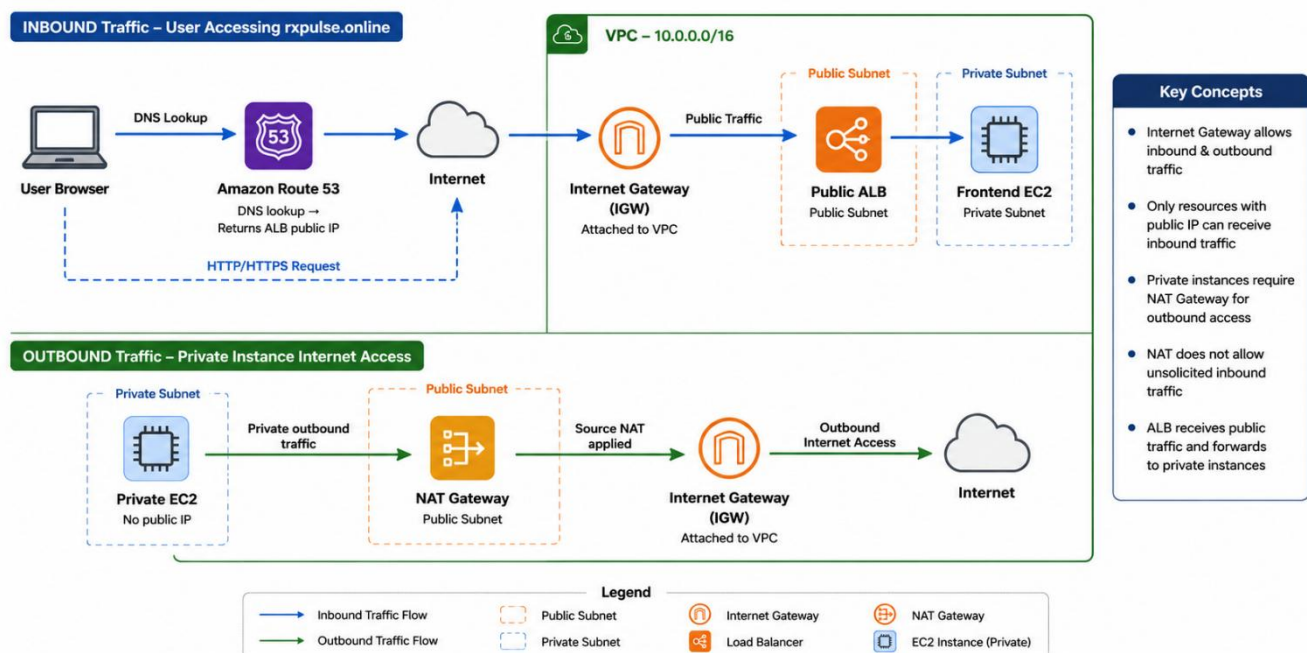
The IGW has no bandwidth limit — it scales with your traffic automatically.

5.2 How the Internet Gateway Enables Public Access

The Internet Gateway alone does not make a subnet 'public'. A subnet becomes public only when it has a route table entry pointing to the IGW for all internet-bound traffic (0.0.0.0/0). The combination of IGW + route table entry + public IP on the instance = full internet connectivity.

5.3 Traffic Flow Through the Internet Gateway

5.3 – Traffic Flow Through the Internet Gateway



AWS Console Implementation Steps

Step 1: Navigate to VPC > Internet Gateways > Create Internet Gateway

Step 2: Name: production-igw

Step 3: Click 'Create Internet Gateway'

Step 4: Select the IGW > Actions > Attach to VPC

Step 5: Select production-vpc > Click 'Attach Internet Gateway'

Chapter 6: NAT Gateway

A NAT Gateway (Network Address Translation Gateway) allows instances in private subnets to initiate outbound connections to the internet — for example, to download software packages — while preventing the internet from initiating inbound connections to those instances. This is critical for the security of our private EC2 instances.

6.1 Why Private Instances Need NAT

Our frontend EC2 instances are in private subnets with no public IP addresses. However, they still need to communicate outbound to the internet for:

- Downloading system packages: `apt-get install apache2, nginx`
- Downloading application code from GitHub
- Fetching Node.js packages (`npm install`)
- Pulling Docker images
- Software updates and security patches

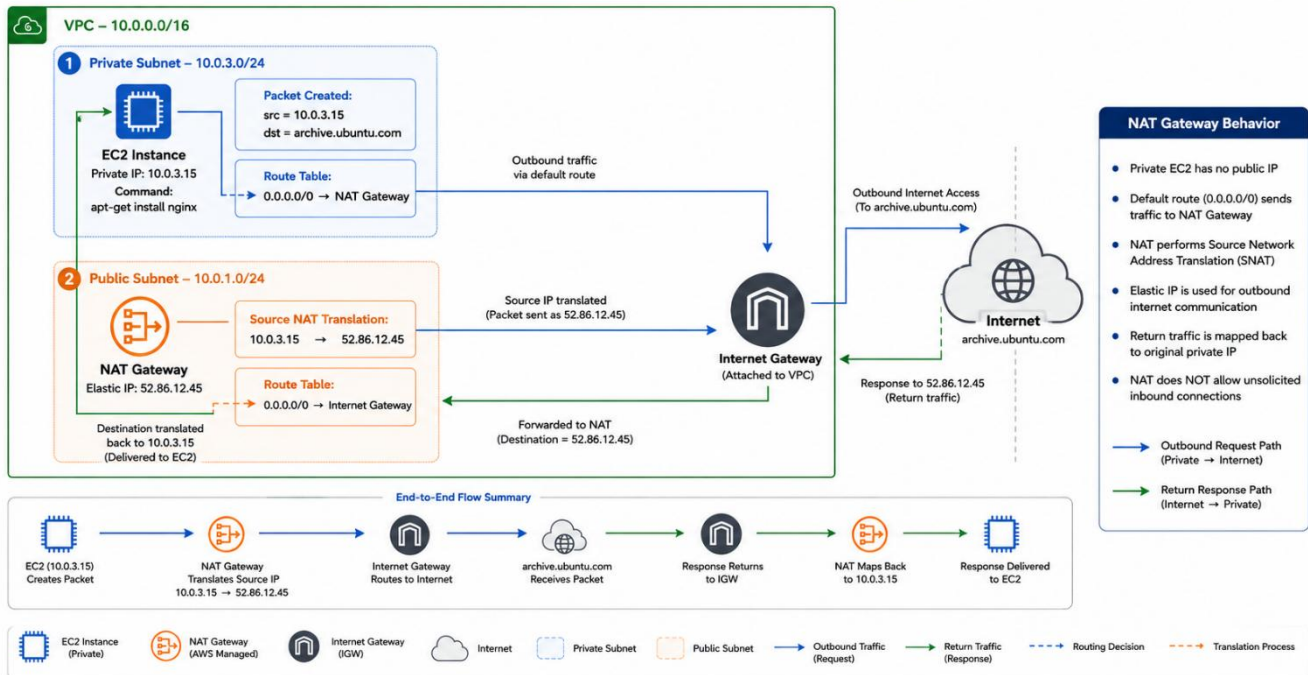
Without NAT Gateway, private instances have zero internet access — no outbound and no inbound. NAT Gateway enables controlled outbound access while maintaining the security of no inbound internet access.

6.2 How NAT Gateway Works

The NAT Gateway sits in a public subnet and has an Elastic IP address (a static public IP). When a private instance sends a request to the internet, the request goes to the NAT Gateway, which replaces the private source IP (e.g., 10.0.3.15) with its own public Elastic IP address. The response from the internet comes back to the NAT Gateway's public IP, and the NAT Gateway forwards it to the original private instance. The internet never sees the private instance's IP address.

6.3 NAT Gateway Traffic Flow Diagram

6.3 – NAT Gateway Traffic Flow Diagram



6.4 NAT Gateway High Availability Design

A single NAT Gateway is associated with one AZ. For true High Availability, enterprise architectures deploy one NAT Gateway per AZ. In our training setup, we deploy one NAT Gateway in public-subnet-1a. For production grade HA, you would also deploy one in public-subnet-1b.

Design	Architecture
Training (Single NAT)	1 NAT Gateway in us-east-1a. Private subnets in both AZs route through it. Cost-effective for learning.
Production HA (Recommended)	1 NAT Gateway per AZ. Each AZ's private subnets use their local NAT Gateway. If AZ-1a fails, AZ-1b instances use their own NAT Gateway.

AWS Console Implementation Steps

- Step 1: Navigate to VPC > NAT Gateways > Create NAT Gateway
- Step 2: Name: production-nat-gateway
- Step 3: Subnet: select public-subnet-1a (NAT Gateway MUST be in a PUBLIC subnet)
- Step 4: Connectivity type: Public
- Step 5: Click 'Allocate Elastic IP' (this assigns a static public IP to the NAT Gateway)
- Step 6: Click 'Create NAT Gateway'
- Step 7: Wait for status to become 'Available' (takes 1-2 minutes)

Chapter 7: Route Tables

A Route Table is a set of rules (called routes) that determine where network traffic is directed within your VPC. Every subnet must be associated with a route table. Understanding route tables is essential because they are what make subnets 'public' or 'private'.

7.1 How Route Tables Work

When an EC2 instance sends network traffic, the VPC looks up the destination IP address in the associated route table and forwards the traffic to the specified target. Routes are evaluated in order of most specific to least specific (longest prefix match).

7.2 Our Route Table Configuration

7.2.1 Public Route Table (for public subnets)

Destination	Target
10.0.0.0/16	local — all internal VPC traffic stays inside the VPC
0.0.0.0/0	igw-xxxxxxxxxx (Internet Gateway) — all other traffic goes to internet

This route table makes any subnet public: the 0.0.0.0/0 → IGW route means that traffic destined for any non-VPC address is sent to the Internet Gateway for internet access.

7.2.2 Private Route Table (for private subnets)

Destination	Target
10.0.0.0/16	local — all internal VPC traffic stays inside the VPC
0.0.0.0/0	nat-xxxxxxxxxx (NAT Gateway) — outbound internet via NAT

This route table is associated with all private subnets (frontend, backend, DB). Instances can initiate outbound internet connections via NAT Gateway, but the internet cannot initiate inbound connections to them.

7.3 Route Table Association

A route table association connects a subnet to a route table. One subnet can only be associated with one route table at a time. However, one route table can be associated with multiple subnets. In our architecture:

- Public Route Table is associated with: public-subnet-1a, public-subnet-1b
- Private Route Table is associated with: frontend-private-1a, frontend-private-1b, backend-private-1a, backend-private-1b, db-private-1a, db-private-1b

AWS Console Implementation Steps

Step 1: Navigate to VPC > Route Tables > Create route table

Step 2: Name: public-route-table, VPC: production-vpc

Step 3: After creation, select it > Routes tab > Edit routes

Step 4: Add route: Destination 0.0.0.0/0, Target: Internet Gateway (select your IGW)

Step 5: Subnet Associations tab > Edit subnet associations > select public-subnet-1a and public-subnet-1b

Step 6: Create another route table named: private-route-table

Step 7: Add route: Destination 0.0.0.0/0, Target: NAT Gateway (select your NAT GW)

Step 8: Associate all 6 private subnets with the private-route-table

Chapter 8: Elastic IP Address

An Elastic IP (EIP) address is a static, public IPv4 address designed for dynamic cloud computing. Unlike a regular public IP that changes every time you stop and start an instance, an Elastic IP remains constant — it is 'elastic' in that you can attach and detach it from resources as needed.

8.1 Why Elastic IP is Needed

- NAT Gateway requires a static public IP so that outbound traffic from private instances always appears to come from the same address. This is important for IP whitelisting at external services.
- If you have a server that must always be reachable at the same IP (e.g., a mail server, a VPN endpoint, or a Bastion Host that has been whitelisted), you attach an Elastic IP.
- Elastic IPs can be quickly remapped: if your server fails, you can detach the EIP and attach it to a replacement server — maintaining the same public IP address and minimising downtime.

8.2 Elastic IP and NAT Gateway

When we created the NAT Gateway in Chapter 6, we clicked 'Allocate Elastic IP'. This allocated a new Elastic IP address from AWS's pool and permanently associated it with our NAT Gateway. All outbound traffic from private instances now appears to come from this Elastic IP.

IMPORTANT: AWS charges for Elastic IPs that are allocated but NOT associated with a running resource.

Cost: ~\$0.005/hour for unused EIPs. Always release EIPs you are not using.

Elastic IPs are region-specific — you cannot use an EIP from us-east-1 in eu-west-1.

Chapter 9: Security Groups

Security Groups are virtual firewalls that control inbound and outbound traffic for AWS resources such as EC2 instances and load balancers. Security Groups are the primary mechanism for controlling who can communicate with what inside your VPC — and they are stateful.

9.1 Stateful Firewall Explained

Security Groups are stateful, meaning that if you allow inbound traffic on a port, the corresponding outbound response is automatically allowed — you do not need a separate outbound rule for the reply. This is different from network ACLs, which are stateless.

Stateful Example: You create an inbound rule allowing TCP port 80 from the internet.

A user sends an HTTP request to your EC2 instance on port 80 → ALLOWED by inbound rule.

Your EC2 sends the HTTP response back to the user → AUTOMATICALLY ALLOWED (stateful).

You do NOT need a separate outbound rule for the response.

This simplifies configuration significantly compared to stateless firewalls.

9.2 Security Group Rule Components

Component	Explanation
Type	Protocol category (e.g., SSH, HTTP, HTTPS, Custom TCP, All Traffic)
Protocol	TCP, UDP, ICMP, or All
Port Range	Single port (e.g., 22) or range (e.g., 8080-8090)
Source (Inbound)	IP range (CIDR), another Security Group ID, or prefix list
Destination (Outbound)	IP range (CIDR), another Security Group ID, or prefix list
Description	Human-readable label for the rule (recommended for documentation)

9.3 Our Security Group Architecture

Our 3-tier architecture implements 6 security groups, each with carefully defined rules that create a layered defence. Traffic can only flow in one permitted direction through each layer.

9.3.1 ALB Security Group (Public-facing)

Rule Type	Details
Inbound — HTTP	Port 80 from 0.0.0.0/0 (all internet users) — to redirect to HTTPS
Inbound — HTTPS	Port 443 from 0.0.0.0/0 (all internet users) — secure web traffic
Outbound	All traffic to Frontend Security Group — ALB forwards to frontend instances

9.3.2 Bastion Host Security Group

Rule Type	Details
Inbound — SSH	Port 22 from YOUR_IP/32 ONLY — never from 0.0.0.0/0
Outbound — SSH	Port 22 to Frontend SG, Backend SG, DB SG — to reach private instances

CRITICAL SECURITY RULE: NEVER allow SSH (port 22) from 0.0.0.0/0 on the Bastion Host.

Allowing SSH from anywhere exposes your server to brute-force attacks from the entire internet.

Always restrict Bastion SSH to your specific IP address or your office's IP range only.

Regularly rotate your SSH key pair and audit who has access.

9.3.3 Frontend Security Group

Rule Type	Details
Inbound — HTTP (80)	From ALB Security Group ID ONLY — not from internet directly
Inbound — HTTPS (443)	From ALB Security Group ID ONLY
Inbound — SSH (22)	From Bastion Host Security Group ID ONLY — for admin access
Outbound	All traffic to Internal ALB Security Group — frontend calls backend

9.3.4 Internal ALB Security Group

Rule Type	Details
Inbound — HTTP (80)	From Frontend Security Group ID ONLY
Inbound — Custom (e.g., 8080)	From Frontend Security Group — for Node.js/API traffic
Outbound	All traffic to Backend Security Group

9.3.5 Backend Security Group

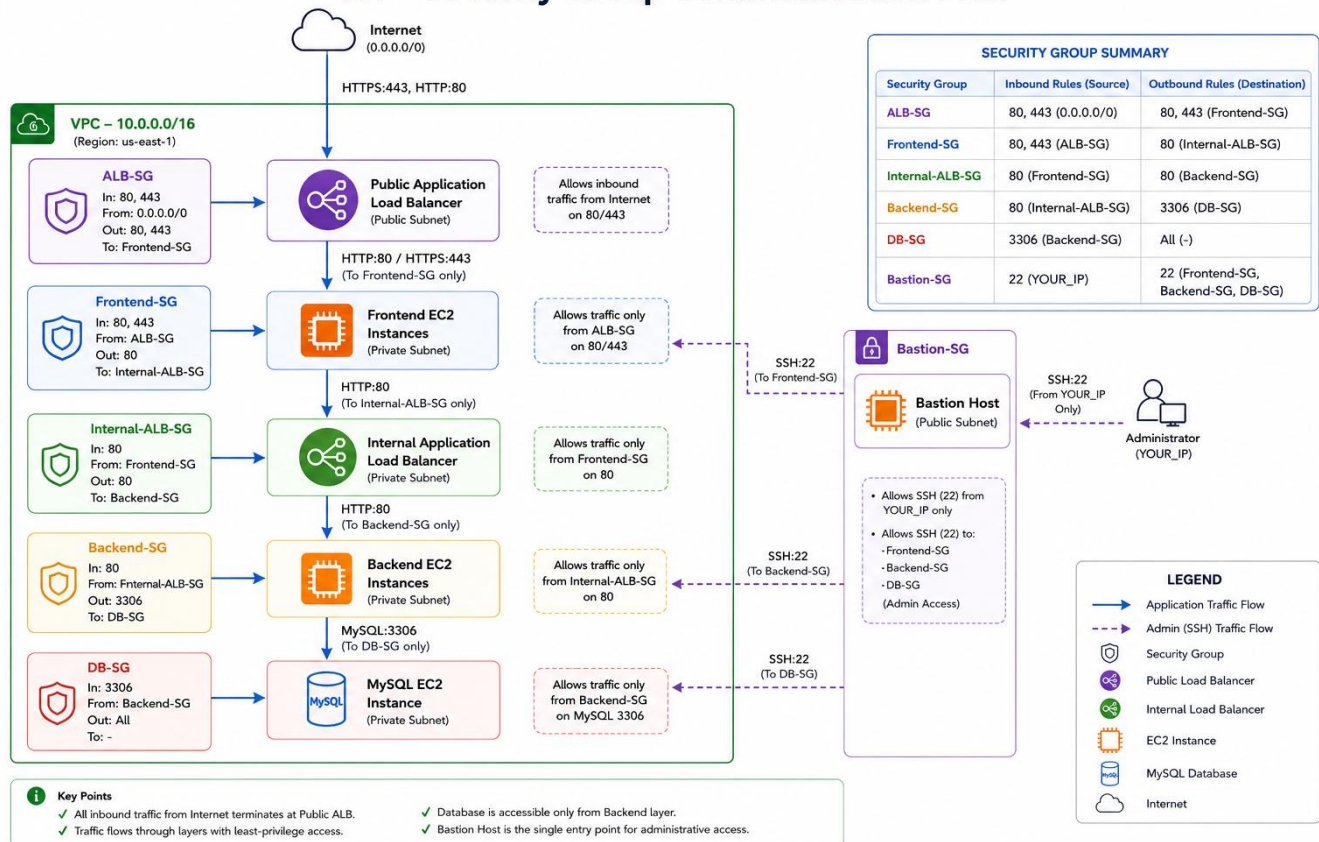
Rule Type	Details
Inbound — Custom (3000, 5000, 8080)	From Internal ALB Security Group ID ONLY
Inbound — SSH (22)	From Bastion Host Security Group ID ONLY
Outbound	All traffic to Database Security Group (port 3306)

9.3.6 Database Security Group

Rule Type	Details
Inbound — MySQL/Aurora (3306)	From Backend Security Group ID ONLY — nothing else can reach the DB
Inbound — SSH (22)	From Bastion Host Security Group ID ONLY — for DBA access
Outbound	All traffic (responses to backend queries)

9.4 Security Group Communication Flow

9.4 – Security Group Communication Flow



AWS Console Implementation Steps

- Step 1: Navigate to EC2 > Security Groups > Create security group
- Step 2: Name: alb-sg, VPC: production-vpc
- Step 3: Inbound rules: Add HTTP (port 80) from 0.0.0.0/0 and HTTPS (port 443) from 0.0.0.0/0
- Step 4: Create security group for Bastion Host: bastion-sg, inbound SSH from your IP
- Step 5: Create frontend-sg: inbound HTTP/HTTPS from alb-sg ID, SSH from bastion-sg ID
- Step 6: Create internal-alb-sg: inbound HTTP from frontend-sg ID
- Step 7: Create backend-sg: inbound custom port from internal-alb-sg ID, SSH from bastion-sg
- Step 8: Create db-sg: inbound MySQL/Aurora (3306) from backend-sg ID only
- Step 9: Verify each SG references the correct SG ID (not CIDR) for internal rules

Chapter 10: EC2 — Elastic Compute Cloud

Amazon EC2 (Elastic Compute Cloud) is the core compute service of AWS. EC2 provides resizable virtual servers in the cloud, called instances. These instances run your operating system and your applications, just like physical servers — but they are created, configured, and terminated through software in minutes.

10.1 EC2 Instance Types

AWS offers hundreds of instance types optimised for different workloads. Each instance type specifies vCPUs, RAM, storage, and network bandwidth:

Family	Use Case	Examples
t3 / t2 (General Purpose)	Development, small apps, testing, our training	t2.micro (free tier), t3.small, t3.medium
m5 / m6i (General Purpose)	Production web servers, application servers	m5.large, m5.xlarge, m5.2xlarge
c5 / c6i (Compute Optimised)	High-performance compute, CPU-intensive apps	c5.large, c5.4xlarge
r5 / r6i (Memory Optimised)	Databases, in-memory caching, analytics	r5.large, r5.2xlarge

10.2 EC2 Components We Configure

10.2.1 Amazon Machine Image (AMI)

An AMI is a pre-configured template that contains the operating system, application server, and application for your instance. In our training, we use the Ubuntu 22.04 LTS AMI (ami-0e86e20dae9224db8 in us-east-1 — always verify the current AMI ID in the console). Ubuntu is chosen because it is widely used, well-documented, and has excellent package management (apt).

10.2.2 Key Pairs

A Key Pair consists of a public key (stored by AWS) and a private key (downloaded by you). The private key (.pem file) is used to SSH into your Linux EC2 instances. Treat your .pem file like a password — never share it, never commit it to Git, and store it securely.

10.2.3 Networking Configuration for Our Instances

Instance	Subnet SG Public IP
Bastion Host	public-subnet-1a bastion-sg Yes (or Elastic IP)
Frontend EC2 #1	frontend-private-1a frontend-sg No
Frontend EC2 #2	frontend-private-1b frontend-sg No
Backend EC2 #1	backend-private-1a backend-sg No
Backend EC2 #2	backend-private-1b backend-sg No
MySQL DB EC2	db-private-1a db-sg No

AWS Console Implementation Steps

- Step 1: Navigate to EC2 > Instances > Launch Instance
- Step 2: Name your instance (e.g., bastion-host)
- Step 3: Select Ubuntu Server 22.04 LTS AMI
- Step 4: Instance type: t2.micro (free tier eligible) or t3.small
- Step 5: Key pair: create new or select existing .pem key
- Step 6: Network settings: Select production-vpc
- Step 7: Subnet: Select appropriate subnet (e.g., public-subnet-1a for Bastion)
- Step 8: Auto-assign public IP: Enable for Bastion only, Disable for all private instances
- Step 9: Security group: Select the appropriate SG created in Chapter 9
- Step 10: Configure storage: 8 GB gp3 is sufficient for our training
- Step 11: Advanced Details > User data: paste your setup script (see Chapter 12)
- Step 12: Click 'Launch Instance'

Chapter 11: Bastion Host

A Bastion Host (also called a Jump Server or Jump Box) is a special-purpose EC2 instance that acts as the single secure entry point for SSH administrative access to all private instances in your VPC. It is the only instance that has a public IP address and allows inbound SSH from the internet (restricted to specific IPs).

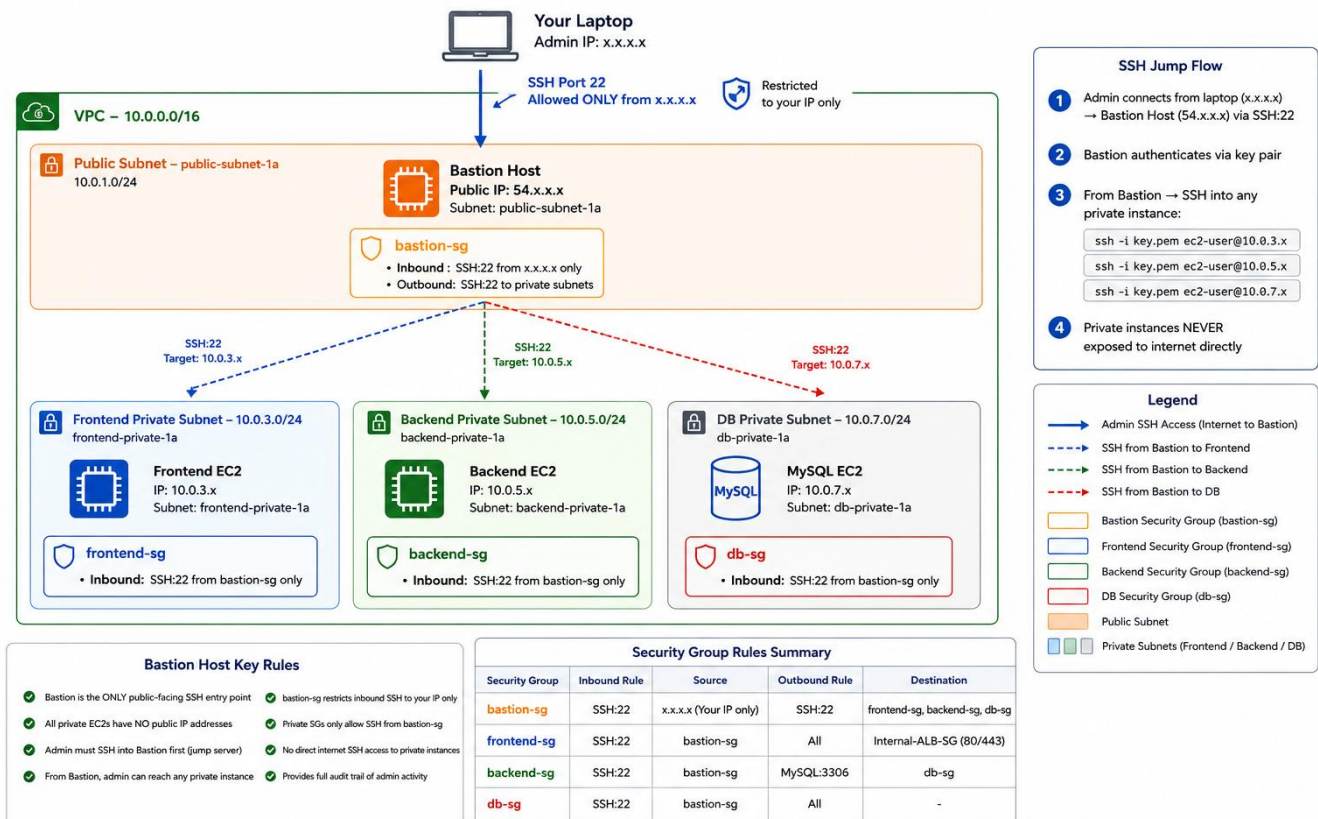
11.1 Why a Bastion Host is Needed

Our frontend, backend, and database EC2 instances are in private subnets with no public IP addresses and no inbound internet routes. This is a security best practice — they should never be directly accessible from the internet. However, administrators need to SSH into these instances to deploy code, debug issues, and perform maintenance.

The solution is the Bastion Host. Administrators SSH into the Bastion Host first (the only publicly accessible server), and from there, SSH into any private instance. This creates a single, auditable, and controlled entry point.

11.2 Bastion Host SSH Architecture

11.2 – Bastion Host SSH Architecture



11.3 SSH Connection Commands

From your laptop → Bastion Host:

```
ssh -i your-key.pem ubuntu@<bastion-public-ip>
```

From Bastion Host → Frontend EC2:

```
ssh -i your-key.pem ubuntu@10.0.3.x
```

SSH Agent Forwarding (recommended — avoids copying key to Bastion):

```
ssh -i your-key.pem -A ubuntu@<bastion-public-ip>
```

Then: `ssh ubuntu@10.0.3.x` (key is forwarded from your laptop)

11.4 Bastion Host Best Practices

- Use a minimal AMI — install nothing extra on the Bastion Host to reduce attack surface.
- Enable CloudWatch Logs for SSH session logging (audit trail).
- Restrict SSH to your organisation's IP range or use AWS Systems Manager Session Manager instead for zero-open-port access.
- Always use key-based authentication — never password authentication.
- Regularly rotate key pairs and review who has access.
- Consider placing the Bastion Host behind a Network Load Balancer for redundancy.

Chapter 12: Application Deployment and User Data Scripts

User Data is a feature of EC2 that allows you to provide a shell script that runs automatically when an instance first boots. This is used to automate the installation and configuration of software, making deployments repeatable and consistent.

12.1 Why User Data is Important

- **Automation:** No manual SSH required for initial setup.
- **Repeatability:** Every instance launched from the same template runs the exact same setup script.
- **Auto Scaling Integration:** When Auto Scaling Groups launch new instances, User Data ensures they are configured correctly without human intervention.
- **Infrastructure as Code:** User Data scripts are the simplest form of automated infrastructure configuration.

12.2 Applications Used in Our Architecture

Application	Layer / Purpose
Apache2 (httpd)	Frontend — simple static web server for HTML pages
Nginx	Frontend — reverse proxy, load balancer, or web server for frontend
Node.js (Express)	Backend — JavaScript runtime for API server
Python Flask	Backend — Python micro web framework for API endpoints
MySQL	Database — relational database for application data

12.3 Frontend User Data Script (Apache + HTML)

Frontend User Data Script

```
#!/bin/bash
# Update package index
apt-get update -y

# Install Apache web server
apt-get install -y apache2

# Start and enable Apache
systemctl start apache2
systemctl enable apache2

# Get instance metadata for display
INSTANCE_ID=$(curl -s http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -s http://169.254.169.254/latest/meta-data/placement/availability-zone)

# Create a simple HTML page showing instance info
cat > /var/www/html/index.html << EOF
<!DOCTYPE html>
<html><head><title>rxpulse.online - HipsterShop</title></head>
<body>
  <h1>Welcome to HipsterShop Frontend</h1>
  <p>Instance ID: $INSTANCE_ID</p>
  <p>Availability Zone: $AZ</p>
</body></html>
EOF
```

12.4 Backend User Data Script (Node.js)

Backend User Data Script

```
#!/bin/bash
apt-get update -y
curl -fsSL https://deb.nodesource.com/setup_18.x | bash -
apt-get install -y nodejs

# Create app directory
mkdir -p /app && cd /app

# Create Node.js Express API server
cat > server.js << 'EOF'
const express = require('express');
const app = express();
app.get('/health', (req, res) => res.json({status:'healthy'}));
app.get('/api/products', (req, res) => res.json([
  {id:1, name:'Vintage Backpack', price:89.99},
  {id:2, name:'Coffee Mug', price:12.99}
]));
app.listen(3000, () => console.log('Backend API running on port 3000'));
EOF

npm init -y && npm install express
node server.js &
```

12.5 Database User Data Script (MySQL)

Database User Data Script

```
#!/bin/bash
apt-get update -y
apt-get install -y mysql-server

# Configure MySQL to listen on all interfaces
sed -i 's/bind-address.*/bind-address = 0.0.0.0/'
/etc/mysql/mysql.conf.d/mysqld.cnf

systemctl start mysql
systemctl enable mysql

# Create database and application user
mysql -e "CREATE DATABASE hipstershop;"
mysql -e "CREATE USER 'appuser'@'%' IDENTIFIED BY 'SecurePassword123!';"
mysql -e "GRANT ALL ON hipstershop.* TO 'appuser'@'%';"
mysql -e "FLUSH PRIVILEGES;"
```

12.6 GitHub Repository Reference

Application Source Code Repository:

<https://github.com/rajcocvs/Three-Tier>

This repository contains:

- Frontend HTML, CSS, JavaScript files
- Backend Node.js / Python Flask application code
- User data scripts for each tier
- Database schema and seed data
- Architecture reference files

Chapter 13: Target Groups

A Target Group is a logical grouping of resources (EC2 instances, IP addresses, or Lambda functions) that a load balancer routes traffic to. The Application Load Balancer uses Target Groups to determine where to send each request. Target Groups also perform health checks to ensure traffic is only sent to healthy targets.

13.1 Target Group Components

Component	Explanation
Target Type	Instances, IP addresses, or Lambda functions. We use 'Instances'.
Protocol	HTTP or HTTPS for ALB. We use HTTP (ALB handles HTTPS termination).
Port	The port on the targets where traffic is forwarded (e.g., 80 for Apache, 3000 for Node.js)
VPC	Must be the same VPC where targets reside
Health Check Protocol	HTTP or HTTPS — ALB sends requests to check target health
Health Check Path	URL path for health check (e.g., / or /health)
Healthy Threshold	Number of consecutive successes to mark target as healthy (default: 5)
Unhealthy Threshold	Number of consecutive failures to mark target as unhealthy (default: 2)
Interval	How often health checks run (default: 30 seconds)

13.2 Our Target Groups

Target Group Name	Purpose Targets Port
frontend-tg	Receives traffic from Public ALB. Targets: Frontend EC2 instances. Port: 80
backend-tg	Receives traffic from Internal ALB. Targets: Backend EC2 instances. Port: 3000 (Node.js)

13.3 Health Check Configuration

Health checks are critical. If a target fails its health check, the ALB stops sending traffic to it. This is how Auto Scaling Groups replace unhealthy instances and how the ALB achieves fault tolerance.

Frontend Target Group Health Check: GET / on port 80 → expects HTTP 200

Backend Target Group Health Check: GET /health on port 3000 → expects HTTP 200

If Apache is not running, the frontend instance fails health check → removed from rotation

When Auto Scaling replaces the instance, it runs User Data → Apache starts → passes health check

AWS Console Implementation Steps

Step 1: Navigate to EC2 > Target Groups > Create target group

Step 2: Target type: Instances

Step 3: Target group name: frontend-tg

Step 4: Protocol: HTTP, Port: 80, VPC: production-vpc

Step 5: Health checks: Protocol HTTP, Path: / (or /health for backend)

Step 6: Advanced health check settings: Healthy threshold: 3, Unhealthy threshold: 2, Interval: 30s

Step 7: Click 'Next' > Select frontend EC2 instances > Include as pending

Step 8: Create target group

Step 9: Repeat for backend-tg with port 3000 and path /health

Chapter 14: Application Load Balancer (ALB)

The Application Load Balancer (ALB) is a fully managed Layer 7 load balancer that distributes incoming application traffic across multiple targets — such as EC2 instances — in multiple Availability Zones. It is one of the most important services in our architecture, providing high availability, scalability, SSL termination, and intelligent request routing.

14.1 What is Layer 7 Load Balancing?

Layer 7 refers to the Application Layer of the OSI model — the layer where HTTP, HTTPS, WebSocket, and gRPC protocols operate. Unlike simpler load balancers that only look at IP addresses and ports (Layer 4), an ALB can inspect the contents of HTTP requests — the URL path, the HTTP headers, query strings, and the hostname — and make routing decisions based on that content.

Load Balancer Type	OSI Layer Capability
Application Load Balancer (ALB)	Layer 7 — understands HTTP/HTTPS, can route by URL path, host, header, etc.
Network Load Balancer (NLB)	Layer 4 — routes by IP and TCP port only. Ultra-high performance.
Classic Load Balancer (CLB)	Layer 4+7 — legacy, not recommended for new architectures

14.2 ALB Components

14.2.1 Listeners

A Listener is a process that checks for connection requests on a specific port and protocol. For example:

- HTTP Listener on port 80: Receives HTTP traffic and redirects to HTTPS
- HTTPS Listener on port 443: Receives HTTPS traffic, terminates SSL, and forwards to target group

14.2.2 Listener Rules

Listener Rules evaluate incoming requests and decide which target group to forward them to. Rules consist of conditions (IF) and actions (THEN). Rules are evaluated in priority order. The default rule (lowest priority) is the catch-all.

14.3 ALB High Availability

An ALB is inherently highly available. When you create an ALB and select multiple subnets in different AZs, the ALB automatically distributes load across all healthy targets in all selected AZs. If one AZ becomes unavailable, the ALB automatically routes all traffic to targets in the remaining AZs. You never need to manage multiple ALB nodes — AWS does this automatically.

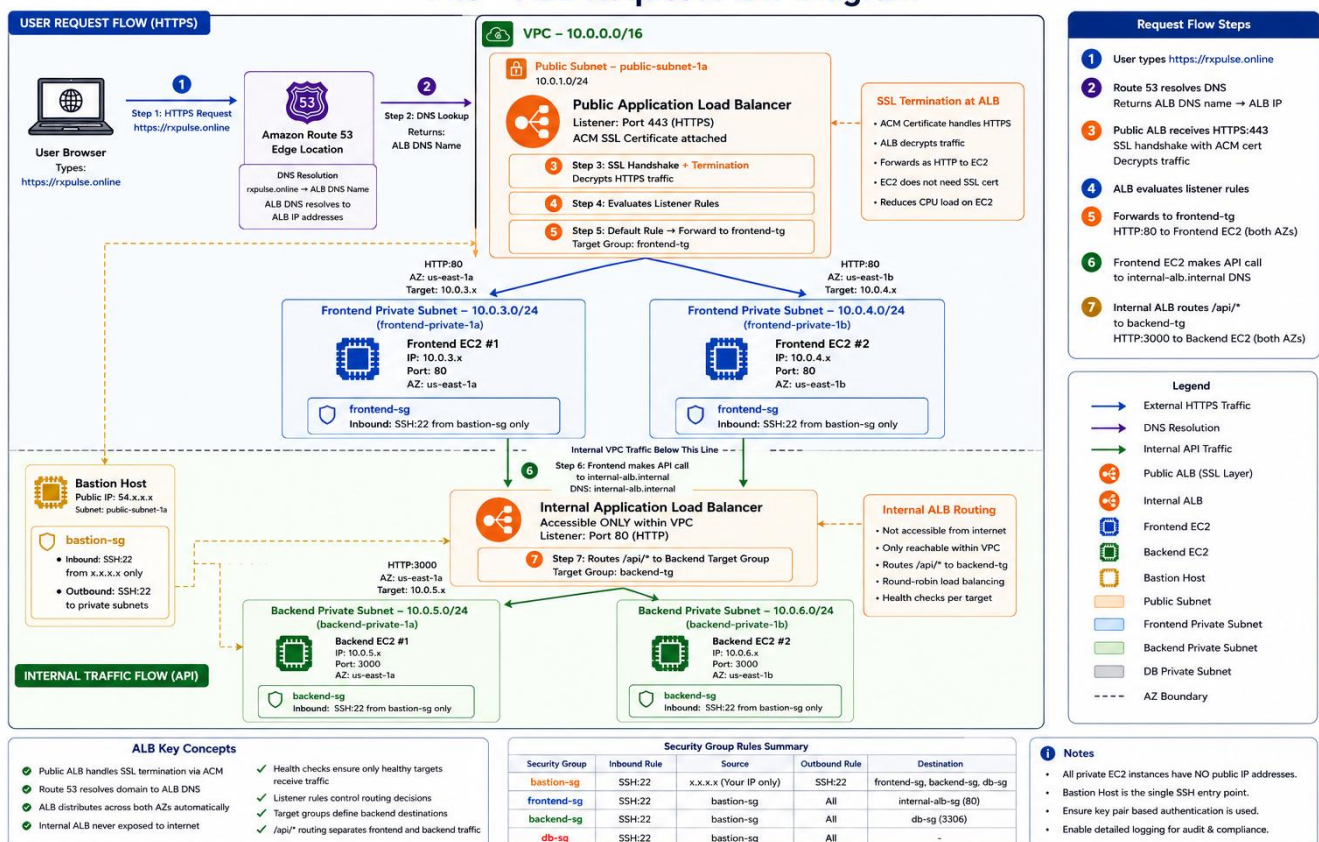
14.4 SSL Termination

Our ALB is configured to terminate SSL. This means:

- The user's HTTPS connection terminates at the ALB — the SSL handshake, certificate validation, and decryption happen at the ALB.
- The ALB forwards decrypted HTTP traffic to the frontend EC2 instances on port 80.
- The private network between the ALB and EC2 instances is isolated within the VPC (trusted zone), so HTTP inside the VPC is acceptable.
- This offloads the computational cost of SSL from EC2 instances and centralises certificate management.

14.5 ALB Request Flow Diagram

14.5 – ALB Request Flow Diagram



AWS Console Implementation Steps

- Step 1: Navigate to EC2 > Load Balancers > Create load balancer
- Step 2: Choose: Application Load Balancer
- Step 3: Name: production-alb, Scheme: Internet-facing, IP type: IPv4
- Step 4: Network mapping: Select production-vpc
- Step 5: Select both AZs: us-east-1a (public-subnet-1a) and us-east-1b (public-subnet-1b)
- Step 6: Security groups: Select alb-sg
- Step 7: Listeners: Add listener HTTP:80 and HTTPS:443
- Step 8: For HTTPS listener: Select SSL certificate from ACM (see Chapter 21)
- Step 9: Default action for HTTPS: Forward to frontend-tg
- Step 10: HTTP listener: Add rule to redirect HTTP to HTTPS (code 301)
- Step 11: Click 'Create load balancer'
- Step 12: Wait for ALB to become 'Active' status

Chapter 15: Internal Application Load Balancer

The Internal Application Load Balancer serves the same purpose as the public ALB — distributing traffic across backend instances — but it is accessible only from within the VPC. It has no public IP address and cannot be reached from the internet under any circumstances.

15.1 Why the Internal ALB is Needed

When the frontend EC2 instances need to call backend APIs (e.g., to fetch product data from the Node.js server), they need a way to distribute those API calls across multiple backend instances and automatically route around any unhealthy backend instance.

Without an Internal ALB, the frontend would need to hardcode backend EC2 IP addresses — which change when Auto Scaling replaces instances. The Internal ALB provides a stable DNS endpoint for the backend, handles load distribution, and performs health checks — all within the private network.

15.2 Internal ALB Configuration

Parameter	Value
Name	internal-alb (or production-internal-alb)
Scheme	Internal — no public IP, not internet-accessible
Subnets	backend-private-1a and backend-private-1b (private subnets)
Security Group	internal-alb-sg
Listener	HTTP:80 (or custom port for API)
Default Action	Forward to backend-tg

15.3 Internal ALB DNS Resolution

AWS automatically assigns a DNS name to every ALB. For the internal ALB, this DNS name resolves only within the VPC. The frontend EC2 instances use this DNS name to call backend APIs. Example: `internal-alb-1234567890.us-east-1.elb.amazonaws.com` — this name resolves to private IP addresses within your VPC.

AWS Console Implementation Steps

- Step 1: Navigate to EC2 > Load Balancers > Create load balancer
- Step 2: Choose: Application Load Balancer
- Step 3: Name: production-internal-alb
- Step 4: Scheme: Internal (IMPORTANT — this makes it private)
- Step 5: Network mapping: Select production-vpc
- Step 6: Select AZs: us-east-1a (backend-private-1a) and us-east-1b (backend-private-1b)
- Step 7: Security group: internal-alb-sg
- Step 8: Listener: HTTP:80, Default action: Forward to backend-tg
- Step 9: Click 'Create load balancer'
- Step 10: Copy the DNS name of the Internal ALB — frontend will use this to call the backend

Chapter 16: ALB Routing Types

One of the most powerful features of the Application Load Balancer is its ability to route traffic intelligently based on the content of HTTP requests. During training, each routing type was implemented separately using dedicated ALBs and EC2 instances to demonstrate the concepts clearly. Understanding these routing types is essential for modern microservices architectures.

Training Note: Each routing type below was demonstrated separately for learning clarity.

In a real production environment, multiple routing rules are combined on a single ALB.

The demonstration used separate Ubuntu EC2 instances with User Data to show different responses based on which routing rule matched, making it clear how each rule works.

16.1 Path-Based Routing

Definition

Path-based routing routes requests to different target groups based on the URL path. The ALB examines the path component of the URL (the part after the domain name) and matches it against listener rules.

Real-World Use Case

A single domain serves multiple microservices: rxpulse.online/ serves the frontend, rxpulse.online/api serves the backend API, and rxpulse.online/admin serves the admin panel — all from one ALB, routed to different target groups.

URL Path	Target Group Service
/	frontend-tg — serves the React/HTML frontend
/api/*	backend-tg — routes to Node.js API servers
/admin/*	admin-tg — routes to admin panel service
/static/*	static-tg — routes to static file server

AWS Console Implementation Steps

Step 1: Go to ALB > Listeners > View/edit rules for HTTPS:443

Step 2: Click 'Add rule' > Add condition: Path

Step 3: Path pattern: /api/*

Step 4: Action: Forward to backend-tg

Step 5: Priority: 1 (evaluated before the default rule)

Step 6: Add another rule: Path / → frontend-tg (priority 2)

Step 7: Default rule: 404 fixed response (catch-all)

16.2 Host-Based Routing

Definition

Host-based routing routes requests based on the HTTP Host header — the domain name in the URL. This allows a single ALB to serve multiple websites or applications on different subdomain names.

Real-World Use Case

Host Header	Target Group Service
rxpulse.online	frontend-tg — main customer-facing website
admin.rxpulse.online	admin-tg — admin dashboard (restricted access)
api.rxpulse.online	backend-tg — public API for mobile apps

AWS Console Implementation Steps

- Step 1: Go to ALB > Listeners > Add rule
- Step 2: Add condition: Host header
- Step 3: Value: admin.rxpulse.online
- Step 4: Action: Forward to admin-tg
- Step 5: Ensure admin.rxpulse.online has a DNS record pointing to the ALB

16.3 Query String Routing

Definition

Query string routing routes requests based on query parameters in the URL — the key=value pairs after the ? in the URL. This is useful for A/B testing, blue/green deployments, or environment routing.

Query String	Routing Decision
?env=prod	Forward to production-tg (stable release)
?env=staging	Forward to staging-tg (new features being tested)
?version=v2	Forward to v2-tg (new API version)

Training Example

URL: `http://alb-dns-name/?env=prod` → routes to one Ubuntu EC2 showing 'Production Environment'

URL: `http://alb-dns-name/?env=staging` → routes to different EC2 showing 'Staging Environment'

This demonstrates how a single ALB can route canary or A/B test traffic without DNS changes.

16.4 HTTP Header-Based Routing

HTTP header routing routes requests based on custom HTTP headers. Mobile apps, API clients, or internal services can set specific headers to be routed to appropriate backend services.

HTTP Header	Value Routing Decision
X-Platform	mobile → mobile-optimised backend target group
X-API-Version	v2 → new API version target group
X-Internal	true → internal service target group

16.5 Source IP-Based Routing

Source IP routing routes requests based on the source IP address of the client. This is useful for routing internal corporate traffic differently from public traffic, or for geographic restrictions.

Source IP Range	Routing Decision
10.0.0.0/8	internal-tg — corporate internal network
192.168.0.0/16	vpn-tg — VPN-connected users
0.0.0.0/0 (default)	public-tg — all other traffic

Chapter 17: Auto Scaling — Launch Templates and Auto Scaling Groups

Auto Scaling is the mechanism that makes your infrastructure elastic and self-healing. It automatically adds EC2 instances when demand increases and removes them when demand decreases, while ensuring that a minimum number of healthy instances are always running.

17.1 Why Auto Scaling is Needed

Problem Without Auto Scaling	How Auto Scaling Solves It
Traffic spike crashes servers because capacity is fixed	ASG adds instances automatically when CPU > 70%
Paying for idle servers during low-traffic periods	ASG removes instances at night when traffic is low
Instance failure takes down the application	ASG detects failed instance, terminates it, launches replacement
Manual scaling requires human intervention and time	Scaling happens in minutes automatically
Deploying code updates requires manual coordination	Launch Template updates propagate to all new instances

17.2 Launch Templates

A Launch Template is a saved configuration for EC2 instances. It stores all the settings needed to launch an instance: AMI, instance type, key pair, security groups, subnet, storage, and User Data. The Auto Scaling Group uses the Launch Template to launch new instances whenever scaling is needed.

Our Launch Template Configuration

Parameter	Value
AMI	Ubuntu 22.04 LTS (latest in us-east-1)
Instance type	t3.small (or t2.micro for free tier training)
Key pair	your-keypair.pem
Security group	frontend-sg (for frontend template) or backend-sg (for backend template)
Storage	8 GB gp3
User Data	Frontend or backend setup script (Chapter 12)

AWS Console Implementation Steps

- Step 1: Navigate to EC2 > Launch Templates > Create launch template
- Step 2: Name: frontend-launch-template, Version description: v1
- Step 3: AMI: Search 'Ubuntu 22.04' → select Ubuntu Server 22.04 LTS
- Step 4: Instance type: t3.small
- Step 5: Key pair: Select your existing key pair
- Step 6: Subnet: Do NOT specify subnet in Launch Template (ASG handles subnet selection)
- Step 7: Security groups: Select frontend-sg
- Step 8: Storage: 8 GiB gp3
- Step 9: Advanced details > User data: Paste the frontend User Data script
- Step 10: Click 'Create launch template'
- Step 11: Repeat for backend-launch-template using backend-sg and backend User Data

17.3 Auto Scaling Groups (ASG)

An Auto Scaling Group is a collection of EC2 instances managed as a unit. The ASG uses a Launch Template to create instances, distributes them across multiple AZs, monitors their health, and replaces unhealthy instances automatically.

17.3.1 ASG Capacity Settings

Setting	Meaning Our Training Value
Desired Capacity	The number of instances the ASG tries to maintain at all times. Training: 2 (one per AZ)
Minimum Capacity	The ASG will never scale below this number. Training: 1
Maximum Capacity	The ASG will never scale above this number. Training: 4

The ASG always tries to maintain 'Desired' instances. If one is terminated (failure or scale-in), a new one is launched to replace it. This is the self-healing behaviour.

17.3.2 Scaling Policies

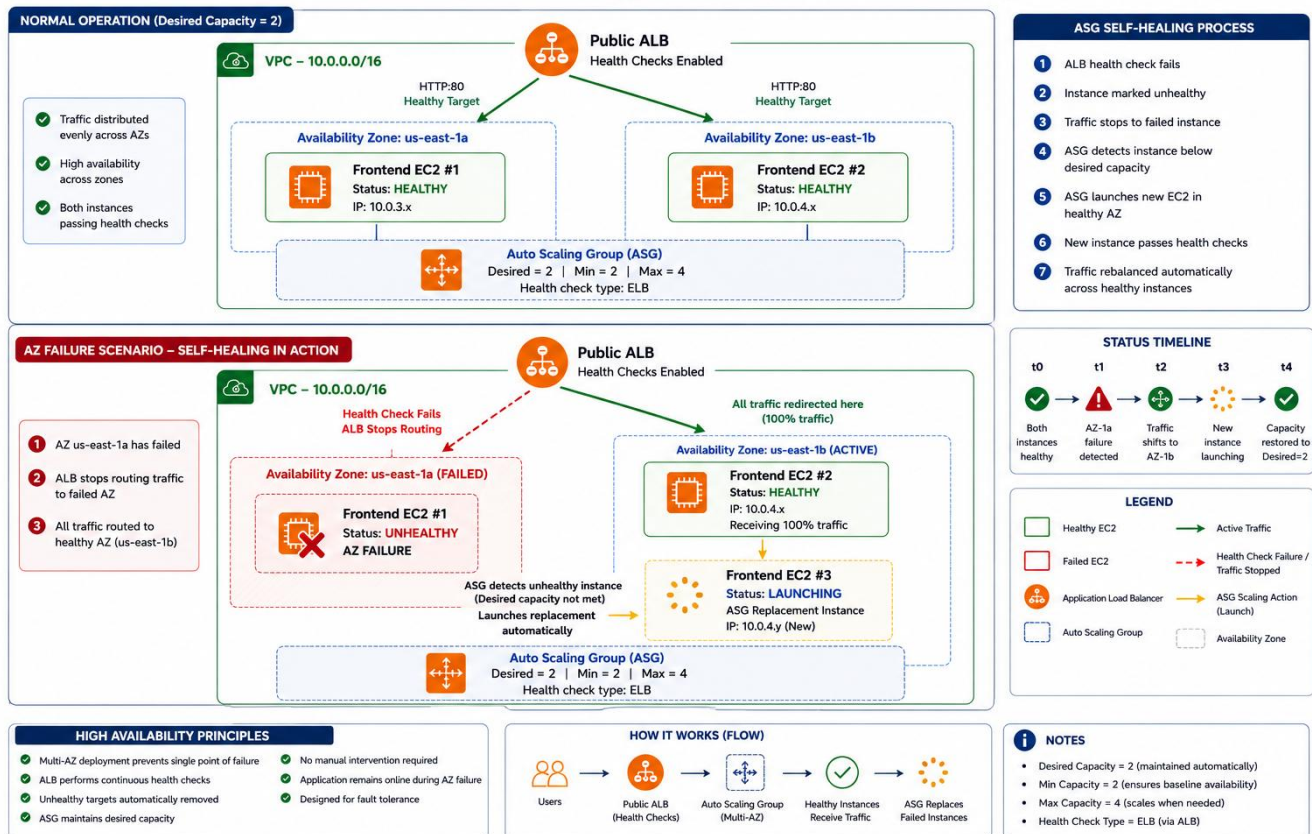
Policy Type	Behaviour
Target Tracking Scaling	Maintains a target metric (e.g., average CPU at 50%). ASG automatically scales in/out. Easiest to configure.
Step Scaling	Scales by a specified number of instances based on CloudWatch alarm thresholds.
Scheduled Scaling	Scales to specific capacity at specific times (e.g., scale up at 8 AM, scale down at 8 PM).

AWS Console Implementation Steps

- Step 1: Navigate to EC2 > Auto Scaling Groups > Create Auto Scaling group
- Step 2: Name: frontend-asg
- Step 3: Launch template: Select frontend-launch-template, Version: Latest (1)
- Step 4: Network: VPC: production-vpc
- Step 5: AZs and subnets: Select frontend-private-1a and frontend-private-1b
- Step 6: Load balancing: Attach to an existing load balancer
- Step 7: Select: frontend-tg (your existing Target Group)
- Step 8: Health checks: Enable ELB health checks (recommended for ASG + ALB integration)
- Step 9: Desired capacity: 2, Minimum: 1, Maximum: 4
- Step 10: Scaling policies: Target tracking → Average CPU utilisation → Target value: 50%
- Step 11: Click through to 'Create Auto Scaling group'
- Step 12: Repeat for backend-asg using backend-launch-template, backend private subnets, backend-tg

17.4 Multi-AZ ASG Behaviour

17.4 – Multi-AZ Auto Scaling Group (ASG) Self-Healing Behavior



Chapter 18: ACM — AWS Certificate Manager

AWS Certificate Manager (ACM) is a service that lets you provision, manage, and deploy SSL/TLS certificates for use with AWS services. It eliminates the complexity of purchasing, uploading, and renewing SSL certificates — ACM handles it all automatically, at no extra cost.

18.1 SSL/TLS Concepts

SSL (Secure Sockets Layer) and its successor TLS (Transport Layer Security) are cryptographic protocols that provide secure, encrypted communication over a network. When a website uses HTTPS instead of HTTP, it is using TLS encryption.

Why HTTPS is Critical

- **Encryption:** All data transmitted between the user's browser and the server is encrypted. No one can intercept and read it (man-in-the-middle attacks are prevented).
- **Authentication:** The SSL certificate proves to the browser that it is genuinely talking to rxpulse.online and not an imposter.
- **Trust:** Browsers show a padlock icon for HTTPS sites. Users see HTTP sites as 'Not Secure'.
- **SEO:** Google ranks HTTPS sites higher than HTTP sites.
- **Compliance:** PCI-DSS, HIPAA, and most enterprise security standards require HTTPS for data in transit.

18.2 ACM Certificate Types

Certificate Type	Use Case
Public Certificate (free)	For publicly accessible domains. Validated by DNS or email. Used with our ALB.
Private Certificate	For internal use within AWS. Requires AWS Private CA (paid service).
Imported Certificate	Bring your own certificate purchased from a commercial CA.

18.3 DNS Validation with Route 53

When you request a public certificate for rxpulse.online, ACM needs to verify that you own the domain. DNS validation works by asking you to add a specific CNAME record to your domain's DNS. ACM checks that this record exists — proving you control the DNS for the domain. With Route 53, ACM can add this record automatically.

AWS Console Implementation Steps

- Step 1: Navigate to ACM (AWS Certificate Manager) — ensure you are in us-east-1 region
- Step 2: Click 'Request a certificate'
- Step 3: Select 'Request a public certificate' > Next
- Step 4: Domain names: Enter rxpulse.online
- Step 5: Add another name: *.rxpulse.online (wildcard for all subdomains)
- Step 6: Validation method: DNS validation (recommended)
- Step 7: Click 'Request'
- Step 8: On the certificate details page: Click 'Create records in Route 53'
- Step 9: AWS automatically adds the required CNAME validation records to your hosted zone
- Step 10: Wait 5-30 minutes for status to change from 'Pending validation' to 'Issued'
- Step 11: Once issued, the certificate appears in the ALB HTTPS listener configuration

Chapter 19: Route 53 — DNS Management

Amazon Route 53 is AWS's highly available and scalable Domain Name System (DNS) web service. It is named after the DNS protocol port (port 53) and the famous US highway Route 66 — a nod to reliability and connectivity.

19.1 What is DNS?

The Domain Name System is the internet's phone book. When you type rxpulse.online into your browser, your browser does not know the IP address of the server. DNS translates the human-readable domain name (rxpulse.online) into a machine-readable IP address (e.g., 54.x.x.x).

Analogy: DNS is like a phone book.

You look up 'rxpulse.online' → DNS tells you '54.86.123.45'.

Your browser then connects directly to 54.86.123.45.

Without DNS, you would have to remember IP addresses instead of domain names.

DNS is why the internet is usable by humans.

19.2 What is Route 53?

Route 53 is AWS's DNS service. It serves two primary functions:

- **Domain Registration:** You can purchase domain names directly through Route 53 (rxpulse.online was registered here for our training).
- **DNS Hosting:** Route 53 hosts the DNS records for your domain, responding to DNS queries from users around the world via 400+ edge locations.

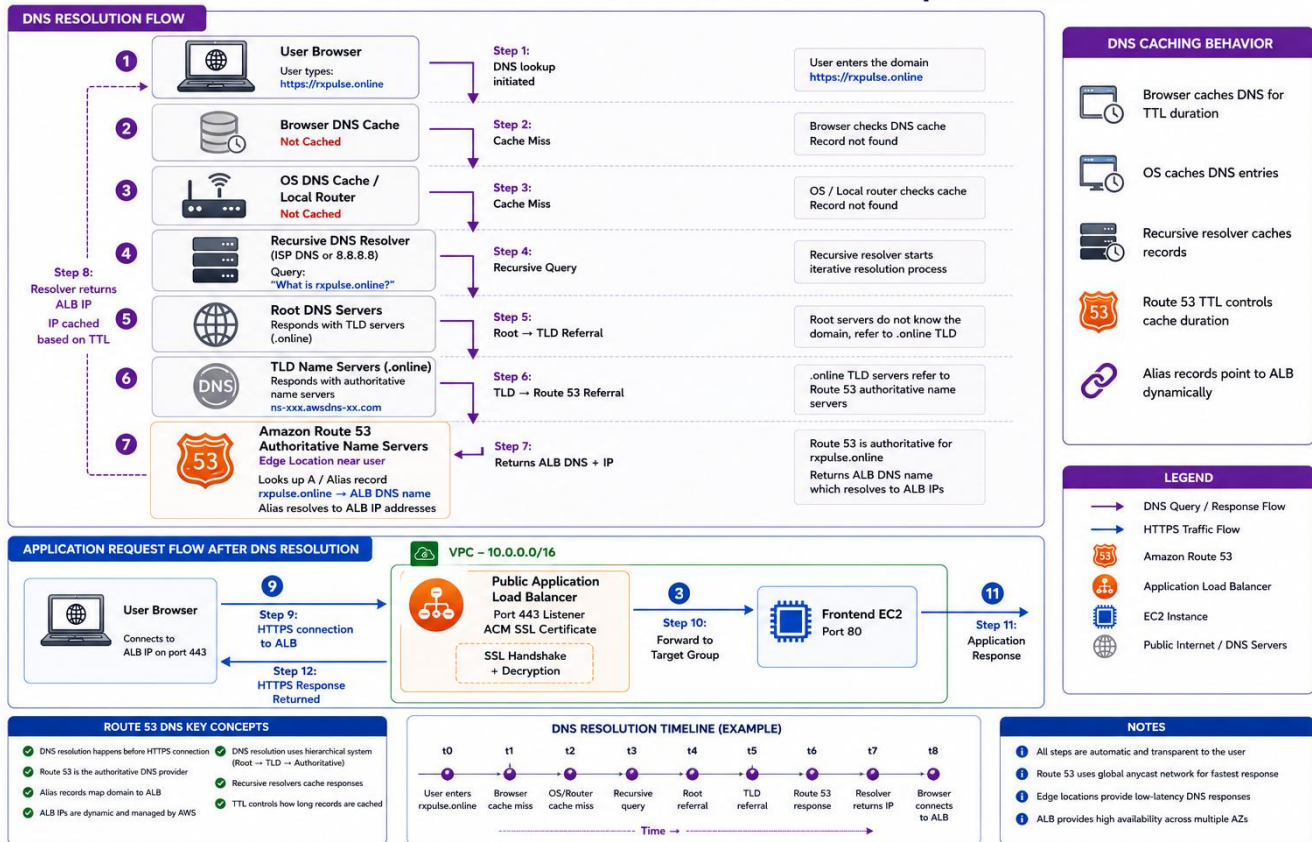
19.3 Hosted Zones

A Hosted Zone is a container in Route 53 that holds DNS records for a specific domain. When you register rxpulse.online, Route 53 automatically creates a Public Hosted Zone for it containing two initial records: NS (Name Server) records and SOA (Start of Authority) record.

Hosted Zone Type	Purpose
Public Hosted Zone	Contains DNS records for a domain accessible from the public internet. Used for rxpulse.online.
Private Hosted Zone	Contains DNS records accessible only within a specified VPC. Used for internal service discovery.

19.4 Route 53 DNS Resolution Flow Diagram

19.4 – Route 53 DNS Resolution Flow for rxpulse.online



19.5 DNS Record Types

19.5.1 A Record (Address Record)

An A Record maps a domain name directly to an IPv4 address. Example: rxpulse.online → 54.86.123.45. However, for ALBs, we use Alias records instead of A records because ALB IP addresses can change.

19.5.2 CNAME Record (Canonical Name Record)

A CNAME record maps one domain name to another domain name (not an IP address). Example: www.rxpulse.online → rxpulse.online. CNAME records cannot be used at the zone apex (the root domain itself, e.g., rxpulse.online) — only Alias records work at the apex.

19.5.3 Alias Record (AWS-Specific)

The Alias record is an AWS extension to DNS. It maps a domain name to an AWS resource (ALB, CloudFront, S3 website, etc.) by DNS name rather than IP. The key advantages are:

- Works at the zone apex (rxpulse.online directly) — CNAME cannot.
- Automatically updates when the ALB's IP addresses change.
- No TTL — changes propagate instantly.
- Free DNS queries for Alias records pointing to AWS resources.

DNS Record	Our rxpulse.online Configuration
rxpulse.online (Alias A)	→ production-alb-xxxxxxx.us-east-1.elb.amazonaws.com
www.rxpulse.online (CNAME)	→ rxpulse.online (or Alias to ALB directly)
admin.rxpulse.online (Alias)	→ production-alb (for host-based routing demo)

AWS Console Implementation Steps

- Step 1: Navigate to Route 53 > Hosted Zones > Select rxpulse.online
- Step 2: Click 'Create record'
- Step 3: Record name: leave blank (for root domain rxpulse.online)
- Step 4: Record type: A
- Step 5: Toggle: Alias ON
- Step 6: Route traffic to: Alias to Application and Classic Load Balancer
- Step 7: Region: us-east-1
- Step 8: Load balancer: Select production-alb
- Step 9: Routing policy: Simple routing
- Step 10: Click 'Create records'
- Step 11: Test: Wait 1-2 minutes, then ping rxpulse.online to verify DNS resolution

Chapter 20: Route 53 Routing Policies

Route 53 offers multiple routing policies that determine how DNS queries are answered. Each policy serves a different use case — from simple single-resource configurations to sophisticated multi-region failover architectures.

20.1 Simple Routing

Simple routing returns a single record value for a DNS query. This is the most basic routing policy — all users get the same answer regardless of their location, the health of the resource, or any other factor.

Real-World Use Case

Pointing rxpulse.online to a single ALB. All users worldwide are directed to the same load balancer. Suitable when you have one region and do not need advanced routing.

Limitation of Simple Routing: If the ALB becomes unavailable, Route 53 continues to return its DNS name even though it is not working. No health checking occurs.
Use Failover Routing if you need automatic failure detection and rerouting.

20.2 Geolocation Routing

Geolocation routing routes users to different resources based on their geographic location — determined by their IP address.

Real-World Use Case

User Location	Routing Decision
United States	→ us-east-1 ALB (lowest latency for US users)
Europe	→ eu-west-1 ALB (GDPR-compliant EU infrastructure)
Asia-Pacific	→ ap-southeast-1 ALB (Singapore region for Asian users)
Default	→ us-east-1 ALB (fallback for all other locations)

Business scenario: rxpulse.online serves pharmaceutical data. EU users must be served by EU infrastructure for GDPR compliance. Geolocation routing enforces this automatically.

20.3 Failover Routing

Failover routing is used for active-passive disaster recovery. A primary resource serves all traffic normally. If Route 53 health checks detect the primary is unhealthy, it automatically routes traffic to the secondary (failover) resource.

Real-World Use Case

Primary: production ALB in us-east-1. Secondary: static S3 website or standby ALB in us-west-2. If us-east-1 goes down, users are automatically redirected to the us-west-2 standby within the DNS TTL period.

Record	Role Health Check
rxpulse.online → us-east-1 ALB	Primary — Health check: HTTPS /health → expects 200
rxpulse.online → us-west-2 ALB	Secondary (Failover) — Activated when primary fails health check

20.4 Multi-Value Answer Routing

Multi-Value routing returns up to 8 IP addresses for a single DNS query, all checked by health checks. Only healthy endpoints are returned. This provides a basic form of load balancing at the DNS level.

Real-World Use Case

Small architectures without a load balancer can use Multi-Value routing to distribute traffic across multiple web server IPs. Each server is health-checked by Route 53, and only healthy server IPs are returned in DNS responses.

Important distinction: Multi-Value routing is NOT a replacement for an ALB. ALBs are stateful, connection-based load balancers. Multi-Value routing is simple DNS round-robin with health checking — much less sophisticated.

Chapter 21: ACM + Route 53 + ALB Integration — Complete HTTPS Architecture

This chapter brings together Route 53, ACM, and the ALB to create a complete, secure HTTPS architecture for rxpulse.online. This is the integration that makes our application accessible securely via a custom domain with a valid SSL certificate.

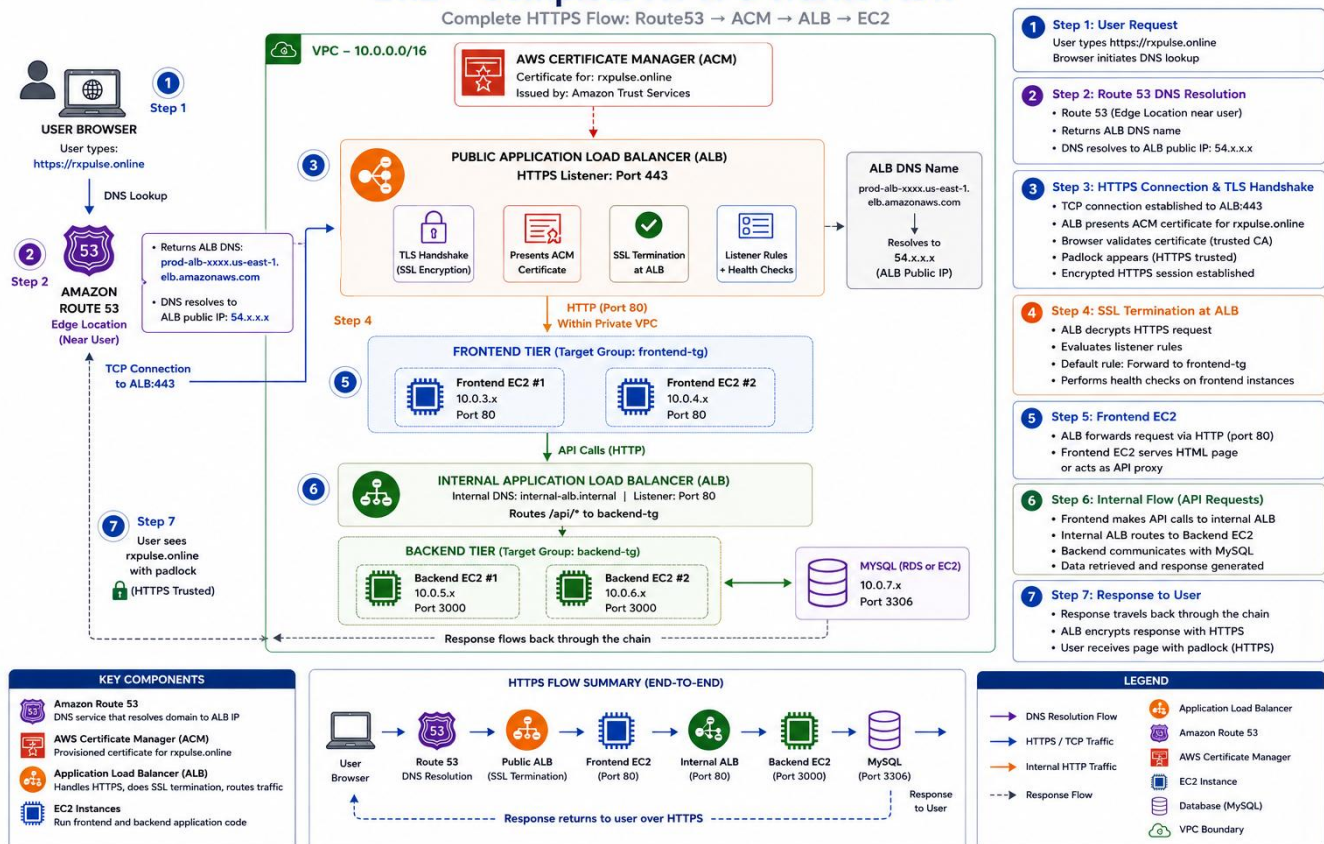
21.1 Component Roles in HTTPS Architecture

Component	Role in HTTPS Flow
Route 53	Translates rxpulse.online to the ALB's DNS name. User's browser gets the ALB IP address.
ACM	Issues and manages the SSL/TLS certificate for rxpulse.online. Attached to ALB HTTPS listener.
ALB HTTPS Listener	Accepts HTTPS connections on port 443, terminates SSL using the ACM certificate, forwards decrypted HTTP to frontend EC2.
HTTP to HTTPS Redirect	ALB HTTP listener (port 80) redirects all traffic to HTTPS (port 443) with 301 redirect.

21.2 Complete HTTPS Traffic Flow

21.2 – Complete HTTPS Traffic Flow

Complete HTTPS Flow: Route53 → ACM → ALB → EC2



21.3 HTTP to HTTPS Redirect Rule

It is essential to redirect all HTTP (port 80) traffic to HTTPS (port 443) so that users who type rxpulse.online (without https://) are automatically upgraded to a secure connection.

AWS Console Implementation Steps

Step 1: Go to ALB > Listeners > Select HTTP:80 listener > View/edit rules

Step 2: Remove the default forward rule

Step 3: Add rule: Action = Redirect

Step 4: Protocol: HTTPS, Port: 443, Status code: 301 (Moved Permanently)

Step 5: This ensures any user who connects via HTTP is automatically redirected to HTTPS

Chapter 22: High Availability Architecture

High Availability (HA) is the design philosophy of building systems that continue to operate without interruption even when individual components fail. In our 3-tier architecture, every layer is designed for high availability using Multi-AZ deployment, redundant instances, and automatic failover mechanisms.

22.1 The 9s of Availability

Availability SLA	Annual Downtime Allowed
99% (Two 9s)	~87.6 hours/year — not suitable for production
99.9% (Three 9s)	~8.7 hours/year — acceptable for some applications
99.99% (Four 9s)	~52 minutes/year — enterprise standard
99.999% (Five 9s)	~5 minutes/year — financial systems, critical infrastructure

22.2 How Each Layer Achieves High Availability

22.2.1 ALB High Availability

The ALB is inherently multi-AZ. AWS deploys ALB nodes in each AZ you select and balances traffic across them. If one AZ's ALB node fails, Route 53 and the ALB infrastructure automatically route traffic through the remaining nodes.

22.2.2 Frontend High Availability

The ASG maintains frontend EC2 instances across us-east-1a and us-east-1b. If AZ-1a fails entirely, the ASG detects unhealthy instances, the ALB stops routing to them, and all traffic is served by the AZ-1b instance. The ASG then launches a replacement instance in AZ-1b to restore desired capacity.

22.2.3 Backend High Availability

Same architecture as frontend. The Internal ALB distributes API calls across backend instances in both AZs. The backend ASG maintains desired capacity and replaces failed instances.

22.2.4 Database High Availability

In our training architecture, the MySQL instance is a single EC2 instance in db-private-1a — a single point of failure. In a production environment, you would use MySQL replication to a standby in db-private-1b, or use Amazon RDS with Multi-AZ deployment which provides automatic failover.

22.3 HA Architecture Summary

High Availability per Tier

HA TIER	MECHANISM	RTO	RPO
DNS	Route 53 (400+ Edge Locations)	Instant	N/A
Public ALB	Multi-AZ ALB nodes	Instant	N/A
Frontend EC2	ASG across 2 AZs	2-5 min	N/A
Internal ALB	Multi-AZ ALB nodes	Instant	N/A
Backend EC2	ASG across 2 AZs	2-5 min	N/A
Database	Single EC2 (training only)	30+ min	Varies
NAT Gateway	Single AZ (training only)	Minutes	N/A

RTO = Recovery Time Objective (how fast you recover)

RPO = Recovery Point Objective (how much data you lose)

Chapter 23: Scalability Architecture

Scalability is the ability of a system to handle growing amounts of work by adding resources. Our architecture is designed to scale horizontally — adding more EC2 instances — rather than vertically, avoiding single points of scale limitation.

23.1 Horizontal vs. Vertical Scaling

Type	Definition Advantages Limitations
Horizontal Scaling (Scale Out)	Adding more instances. Our ASG approach. No downtime. Unlimited scale. Requires load balancer.
Vertical Scaling (Scale Up)	Making one instance bigger. Simple but causes downtime during resize. Has a physical maximum (largest instance type).

23.2 How Our Architecture Scales

23.2.1 Traffic Spike Scenario

Auto Scaling Behaviour During Traffic Spike

```
TIME 09:00 – Normal traffic (2 frontend instances, desired=2):  
[ALB] → [FE-1 in 1a] + [FE-2 in 1b]  
  
TIME 12:00 – Black Friday sale starts. CPU spikes to 85%:  
ASG scaling policy: Target CPU 50% → scale out triggered  
[ALB] → [FE-1 in 1a] + [FE-2 in 1b] + [FE-3 launching] + [FE-4 launching]  
  
TIME 12:05 – New instances pass health checks:  
[ALB] → [FE-1] + [FE-2] + [FE-3] + [FE-4] CPU back to ~50%  
  
TIME 22:00 – Traffic drops. CPU falls to 20%:  
ASG scale-in: Terminates FE-3 and FE-4 (scale-in protection on FE-1&2) |  
[ALB] → [FE-1 in 1a] + [FE-2 in 1b] – back to desired=2
```

23.3 Scalability of Each Component

Component	Scalability Mechanism
Route 53	Globally distributed — scales to millions of DNS queries per second automatically
ACM	Managed service — no scaling required
ALB	Automatically scales nodes — handles millions of concurrent connections
Frontend ASG	Min: 1, Desired: 2, Max: 4 — scales on CPU and request count
Backend ASG	Min: 1, Desired: 2, Max: 4 — scales on CPU and API request count
NAT Gateway	Scales up to 45 Gbps bandwidth automatically
Database	Vertical scaling (increase EC2 size) or migrate to RDS for horizontal read scaling

Chapter 24: Security Architecture — Defence in Depth

Our architecture implements a Defence in Depth security model — multiple layers of security controls that an attacker must breach before reaching sensitive data. Each layer is independent, so compromising one layer does not automatically compromise others.

24.1 Security Layers in Our Architecture

Defence in Depth — 9 Security Layers

```
LAYER 1: Internet Gateway — only allows traffic on ports configured in ALB SG
LAYER 2: ALB Security Group — restricts to HTTP(80) and HTTPS(443) only
LAYER 3: HTTPS Termination — encrypted traffic, certificate authentication
LAYER 4: Frontend SG — accepts HTTP ONLY from ALB SG (not internet directly)
LAYER 5: Private Subnets — no public IPs, no direct internet inbound route
LAYER 6: Internal ALB SG — accepts traffic ONLY from Frontend SG
LAYER 7: Backend SG — accepts traffic ONLY from Internal ALB SG
LAYER 8: Database SG — accepts MySQL ONLY from Backend SG (port 3306)
LAYER 9: Bastion Host — single SSH entry point, restricted to your IP
```

24.2 Network Isolation Architecture

Network isolation is achieved by placing resources in appropriate subnets with appropriate routing:

- Public Subnets: Only Bastion Host and NAT Gateway. These are the only resources that should have any public exposure.
- Frontend Subnets: No public IPs. The ALB forwards traffic to these instances — users never connect directly.
- Backend Subnets: Accessible only from the Internal ALB. Completely isolated from internet and from direct frontend access.
- Database Subnets: The most isolated layer. Only port 3306 from the backend subnet is allowed. No other traffic can reach the database.

24.3 Zero-Trust Traffic Flow

Every communication in our architecture is based on the principle of least privilege — each component can only communicate with what it specifically needs, and nothing more:

- Internet → ALB: Only HTTPS (443) and HTTP (80) for redirect
- ALB → Frontend: Only HTTP (80) — no SSH, no database ports
- Frontend → Internal ALB: Only HTTP/API port
- Internal ALB → Backend: Only application port (3000)
- Backend → Database: Only MySQL port (3306)
- Admin → Bastion: Only SSH (22) from specific IP
- Bastion → Any Private Instance: Only SSH (22)

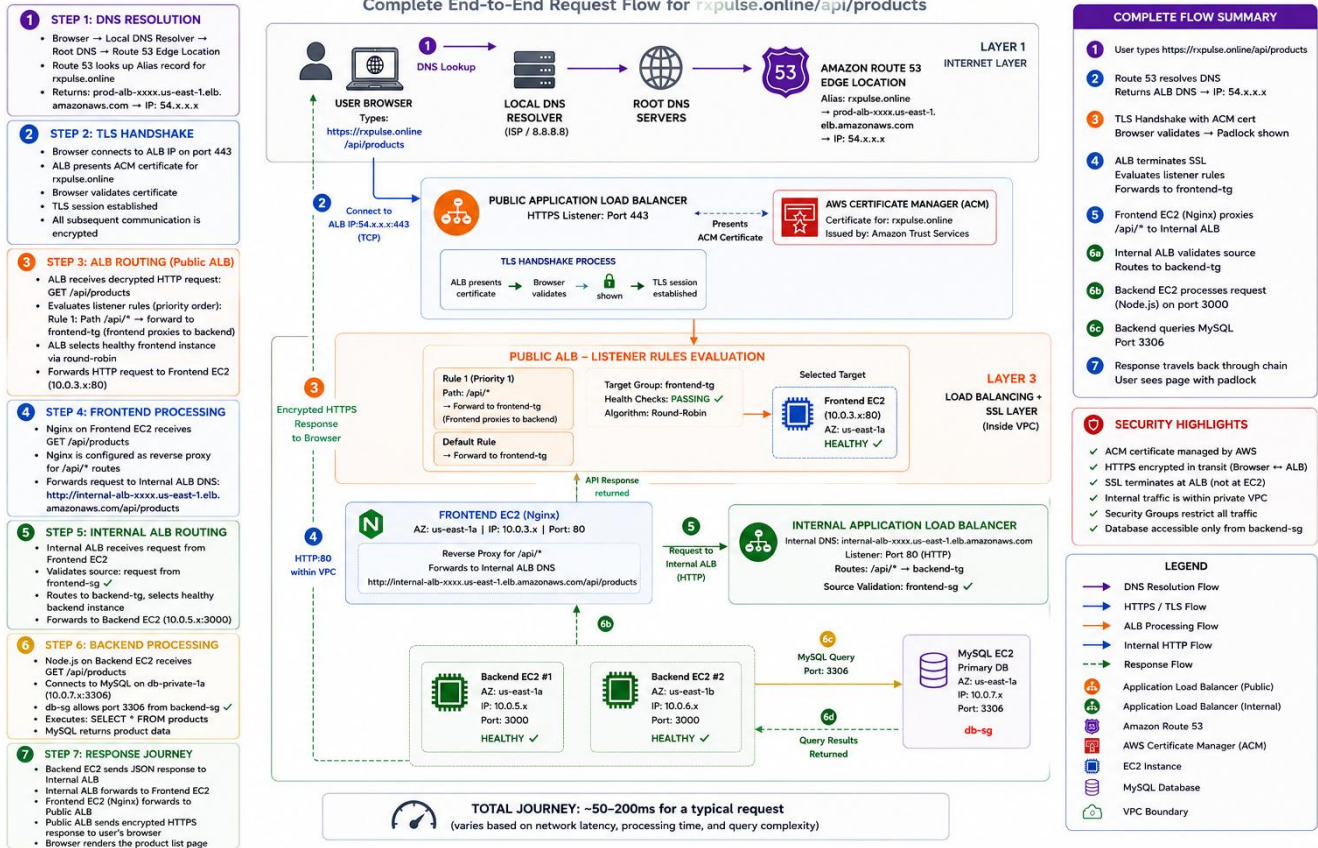
Chapter 25: End-to-End 3-Tier Application Flow

This chapter traces the complete journey of a user request from their browser to the database and back — covering every AWS service, every security group, every routing decision, and every network hop in our production-style architecture.

25.1 Complete Request Lifecycle

25.1 – Complete Request Lifecycle

Complete End-to-End Request Flow for `rxpulse.online/api/products`



25.2 Network Traffic at Each Hop

Network Hop	Source → Destination	Protocol / Port
1. Browser to ALB	User IP → ALB Public IP	HTTPS/443 (encrypted)
2. ALB to Frontend	ALB → Frontend EC2 private IP	HTTP/80 (within VPC)
3. Frontend to Internal ALB	Frontend EC2 → Internal ALB private IP	HTTP/80 or /3000
4. Internal ALB to Backend	Internal ALB → Backend EC2 private IP	HTTP/3000
5. Backend to Database	Backend EC2 → MySQL EC2 private IP	TCP/3306 (MySQL)
6. Response (all hops reversed)	MySQL → Backend → Internal ALB → Frontend → Public ALB → Browser	Same protocols reversed

Chapter 26: Complete Practical AWS Console Implementation Guide

This chapter provides a step-by-step implementation guide for building the entire 3-tier architecture from scratch. Follow these steps in order — each builds on the previous.

26.1 Implementation Order

The order of implementation matters because services depend on each other. Build in this sequence:

Phase	Components to Build
Phase 1: Network Foundation	VPC → Subnets → Internet Gateway → Elastic IP → NAT Gateway → Route Tables
Phase 2: Security	6 Security Groups with correct rules and cross-references
Phase 3: Compute	Bastion Host → Frontend EC2 instances → Backend EC2 instances → MySQL EC2
Phase 4: Load Balancing	Target Groups → Public ALB → Internal ALB
Phase 5: Auto Scaling	Launch Templates → Frontend ASG → Backend ASG
Phase 6: DNS & SSL	ACM Certificate → Route 53 Hosted Zone → DNS Records → HTTPS Listener
Phase 7: Validation	End-to-end testing → Health check verification → DNS resolution testing

26.2 Phase 1: Network Foundation

AWS Console Implementation Steps

- Step 1: Create VPC: VPC > Your VPCs > Create VPC > Name: production-vpc, CIDR: 10.0.0.0/16
- Step 2: Create 8 subnets with correct CIDR blocks and AZ assignments (see Chapter 4 table)
- Step 3: Enable auto-assign IPv4 on BOTH public subnets only
- Step 4: Create Internet Gateway: Name: production-igw > Attach to production-vpc
- Step 5: Allocate Elastic IP: EC2 > Elastic IPs > Allocate Elastic IP Address
- Step 6: Create NAT Gateway: VPC > NAT Gateways > subnet: public-subnet-1a > use allocated EIP
- Step 7: Create public-route-table: add route 0.0.0.0/0 → IGW > associate public subnets
- Step 8: Create private-route-table: add route 0.0.0.0/0 → NAT GW > associate all 6 private subnets
- Step 9: Verify: public subnets have IGW route, private subnets have NAT GW route

26.3 Phase 2: Security Groups

AWS Console Implementation Steps

- Step 1: Create alb-sg: inbound HTTP:80 and HTTPS:443 from 0.0.0.0/0

Step 2: Create bastion-sg: inbound SSH:22 from YOUR_IP_ADDRESS/32 only
Step 3: Create frontend-sg: inbound HTTP:80 from alb-sg, SSH:22 from bastion-sg
Step 4: Create internal-alb-sg: inbound HTTP:80 (or app port) from frontend-sg
Step 5: Create backend-sg: inbound port 3000 from internal-alb-sg, SSH:22 from bastion-sg
Step 6: Create db-sg: inbound MySQL:3306 from backend-sg, SSH:22 from bastion-sg
Step 7: Double-check: each SG references the SG ID (sg-xxxxxxxxx) not a CIDR range for internal rules

26.4 Phase 3: Compute Instances

AWS Console Implementation Steps

Step 1: Create Bastion Host: public-subnet-1a, bastion-sg, enable public IP, Ubuntu 22.04, t2.micro
Step 2: Create Frontend EC2 #1: frontend-private-1a, frontend-sg, NO public IP, User Data: Apache script
Step 3: Create Frontend EC2 #2: frontend-private-1b, frontend-sg, NO public IP, User Data: Apache script
Step 4: Create Backend EC2 #1: backend-private-1a, backend-sg, NO public IP, User Data: Node.js script
Step 5: Create Backend EC2 #2: backend-private-1b, backend-sg, NO public IP, User Data: Node.js script
Step 6: Create MySQL EC2: db-private-1a, db-sg, NO public IP, User Data: MySQL script
Step 7: Test SSH: Laptop → Bastion → Frontend EC2, then curl http://localhost to verify Apache

26.5 Phase 4: Load Balancing

AWS Console Implementation Steps

Step 1: Create Target Group frontend-tg: Instance type, HTTP:80, health check path: /, register both frontend EC2s
Step 2: Create Target Group backend-tg: Instance type, HTTP:3000, health check path: /health, register both backend EC2s
Step 3: Create Public ALB: Internet-facing, both public subnets, alb-sg, HTTP:80 and HTTPS:443 listeners
Step 4: Attach frontend-tg to HTTPS:443 listener as default action
Step 5: HTTP:80 listener: redirect to HTTPS:443
Step 6: Create Internal ALB: Internal scheme, backend private subnets, internal-alb-sg, HTTP:80 listener
Step 7: Attach backend-tg to Internal ALB HTTP:80 listener default action
Step 8: Verify: both ALBs become Active, target groups show healthy targets

26.6 Phase 5: Auto Scaling

AWS Console Implementation Steps

- Step 1: Create Launch Template frontend-launch-template: Ubuntu AMI, t3.small, frontend-sg, frontend User Data
- Step 2: Create Frontend ASG: use frontend-launch-template, select frontend private subnets (both AZs)
- Step 3: Attach Frontend ASG to frontend-tg, enable ELB health checks, Desired:2, Min:1, Max:4
- Step 4: Add Target Tracking scaling policy: Average CPU 50%
- Step 5: Create Launch Template backend-launch-template: Ubuntu AMI, t3.small, backend-sg, backend User Data
- Step 6: Create Backend ASG: backend-launch-template, backend private subnets, attach to backend-tg
- Step 7: Note: You can now terminate the manually created EC2 instances — ASG manages instances
- Step 8: Verify: ASG launches new instances, they appear in Target Groups as healthy

26.7 Phase 6: DNS and SSL

AWS Console Implementation Steps

- Step 1: Navigate to ACM in us-east-1 region > Request certificate > Public certificate
- Step 2: Add domains: rxpulse.online and *.rxpulse.online > DNS validation
- Step 3: Click 'Create records in Route 53' to auto-add validation CNAME
- Step 4: Wait for certificate status: Issued (5-30 minutes)
- Step 5: Navigate to Route 53 > Hosted Zones > rxpulse.online
- Step 6: Create A record (Alias): rxpulse.online → Application Load Balancer → prod-alb
- Step 7: Go to Public ALB > Listeners > HTTPS:443 > Edit > Add ACM certificate
- Step 8: Test: Open browser → https://rxpulse.online → should see padlock + your frontend page

26.8 Phase 7: Validation Checklist

VALIDATION CHECKLIST — verify each item:

- ☐ VPC: 10.0.0.0/16, DNS resolution and hostnames enabled
- ☐ 8 subnets: correct CIDRs and AZ assignments
- ☐ Internet Gateway: attached to VPC
- ☐ NAT Gateway: in public subnet, Elastic IP attached, status Available
- ☐ Route Tables: public subnets → IGW, private subnets → NAT GW
- ☐ 6 Security Groups: correct inbound rules, cross-SG references correct
- ☐ Bastion Host: SSH accessible from your IP, can SSH to private instances
- ☐ Frontend EC2s: Apache running, curl http://localhost returns HTML
- ☐ Backend EC2s: Node.js running, curl http://localhost:3000/health returns OK
- ☐ MySQL EC2: MySQL running, port 3306 accessible from backend instances
- ☐ Target Groups: both showing healthy targets

- [] Public ALB: Active, HTTPS listener with ACM certificate attached
- [] Internal ALB: Active, backend instances healthy
- [] ASG: instances launched and registered in target groups
- [] ACM Certificate: Issued status
- [] Route 53: A record for rxpulse.online pointing to ALB
- [] HTTPS test: <https://rxpulse.online> loads in browser with padlock

Chapter 27: Best Practices and Common Mistakes

27.1 Common Mistakes and How to Avoid Them

COMMON MISTAKE #1: NAT Gateway in a Private Subnet

NAT Gateway MUST be placed in a PUBLIC subnet. If placed in a private subnet, it cannot reach the internet and private instances will have no outbound internet access.

COMMON MISTAKE #2: SSH open to the world (0.0.0.0/0) on Bastion

Never allow SSH from all IP addresses. Always restrict to your specific IP (x.x.x.x/32).
An open SSH port is immediately targeted by automated brute-force scanning tools.

COMMON MISTAKE #3: ACM Certificate in the wrong region

For ALB, request the ACM certificate in the SAME region as the ALB (us-east-1).
For CloudFront, certificates must be in us-east-1 (N. Virginia) regardless of deployment region.

COMMON MISTAKE #4: Target Group with wrong health check path

If your backend runs on port 3000 but health check is configured for port 80, all targets will be marked unhealthy and no traffic will be routed to them.
Always verify: health check port matches the application's listening port.

COMMON MISTAKE #5: Forgetting to associate subnets with route tables

New subnets are automatically associated with the VPC's main (default) route table.
If you forget to explicitly associate your subnets with your custom route tables, private subnets will use whatever routing the main route table has — often incorrect.

27.2 Architecture Best Practices

Networking

- Always plan CIDR blocks before creating the VPC — changing later is difficult.
- Use /24 subnets for each tier to give 254 usable IPs per subnet — sufficient for most applications.
- Reserve IP space: use a VPC CIDR that does not overlap with on-premises networks.
- Place NAT Gateways in each AZ for true HA (one NAT GW per AZ).

Security

- Use Security Group referencing (SG ID as source) not CIDR ranges for internal rules.
- Never assign public IPs to private instances.
- Rotate SSH key pairs regularly.
- Enable VPC Flow Logs to monitor all network traffic.
- Apply the principle of least privilege to all Security Group rules.

Availability and Scaling

- Always deploy across at least 2 Availability Zones for any production workload.
- Set ASG minimum capacity to at least 1 per AZ for true HA.
- Enable ELB health checks on ASG — not just EC2 status checks.
- Test failure scenarios — manually terminate instances and verify the system self-heals.

Cost Optimisation

- Use t3.micro or t3.small for development/training; right-size for production.
- Release unused Elastic IPs — they are charged when unattached.
- Use Auto Scaling to prevent over-provisioning.
- NAT Gateway charges are based on data processed — minimise unnecessary internet traffic from private subnets.

Appendix A: Architecture Quick Reference

A.1 IP Address Reference

Subnet	CIDR	AZ / Purpose
public-subnet-1a	10.0.1.0/24	us-east-1a — Bastion, NAT GW
public-subnet-1b	10.0.2.0/24	us-east-1b — ALB redundancy
frontend-private-1a	10.0.3.0/24	us-east-1a — Frontend EC2
frontend-private-1b	10.0.4.0/24	us-east-1b — Frontend EC2
backend-private-1a	10.0.5.0/24	us-east-1a — Backend EC2
backend-private-1b	10.0.6.0/24	us-east-1b — Backend EC2
db-private-1a	10.0.7.0/24	us-east-1a — MySQL EC2
db-private-1b	10.0.8.0/24	us-east-1b — DB Standby

A.2 Security Group Quick Reference

Security Group	Inbound Rules	Purpose
alb-sg	HTTP:80 and HTTPS:443 from 0.0.0.0/0	Public ALB
bastion-sg	SSH:22 from YOUR_IP/32 only	Bastion Host
frontend-sg	HTTP:80 from alb-sg; SSH:22 from bastion-sg	Frontend EC2
internal-alb-sg	HTTP:80 from frontend-sg	Internal ALB
backend-sg	Port 3000 from internal-alb-sg; SSH from bastion-sg	Backend EC2
db-sg	MySQL:3306 from backend-sg; SSH from bastion-sg	Database EC2

A.3 Port Reference

Port	Protocol / Service
22	SSH — administrative access
80	HTTP — web traffic (redirect to HTTPS)
443	HTTPS — secure web traffic
3000	Node.js Express API server

3306	MySQL database
5000	Python Flask API server
8080	Alternative HTTP port for web services

A.4 AWS Console Navigation Quick Reference

Service	AWS Console Navigation Path
VPC	Services > Networking & Content Delivery > VPC
EC2	Services > Compute > EC2
Security Groups	EC2 > Network & Security > Security Groups
Load Balancers	EC2 > Load Balancing > Load Balancers
Target Groups	EC2 > Load Balancing > Target Groups
Auto Scaling Groups	EC2 > Auto Scaling > Auto Scaling Groups
Route 53	Services > Networking & Content Delivery > Route 53
ACM	Services > Security, Identity, & Compliance > Certificate Manager

A.5 Glossary

Term	Definition
AZ	Availability Zone — a physically separate data centre within an AWS Region
ALB	Application Load Balancer — Layer 7 load balancer that routes HTTP/HTTPS traffic
ASG	Auto Scaling Group — manages a fleet of EC2 instances for HA and elasticity
ACM	AWS Certificate Manager — manages SSL/TLS certificates
CIDR	Classless Inter-Domain Routing — notation for IP address ranges (e.g., 10.0.0.0/16)
DNS	Domain Name System — translates domain names to IP addresses
EIP	Elastic IP — static public IPv4 address allocated to your AWS account
HA	High Availability — design for continued operation despite component failures
IGW	Internet Gateway — connects VPC to the public internet
NAT	Network Address Translation — translates private IPs to public IP for outbound internet access
SG	Security Group — stateful virtual firewall for EC2 instances and ALBs

TLS	Transport Layer Security — cryptographic protocol providing encrypted communication
VPC	Virtual Private Cloud — isolated virtual network within AWS
RTO	Recovery Time Objective — maximum acceptable downtime after a failure
RPO	Recovery Point Objective — maximum acceptable data loss after a failure

— *End of AWS Cloud Documentation & Implementation Guide* —
[Domain: rxpulse.online](#) | [Region: us-east-1](#) | [3-Tier Multi-AZ Architecture](#)